PI_Curs_01

1.1

Bună ziua! În calitate de profesor universitar specializat în acest domeniu, mă bucur să vă ghidez prin elementele fundamentale ale acestei lecții introductive. Obiectivul nostru este să transformăm aceste concepte tehnice într-un set de cunoștințe clare și bine structurate.

Iată Ghidul dumneavoastră de Studiu Complet, bazat strict pe sursele furnizate.

Ghid de Studiu Complet: Introducere. Modele Liniare, Inteligență Artificială și Machine Learning

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Această lecție introductivă stabilește cadrul conceptual al Inteligenței Artificiale (AI) și al subdomeniilor sale cruciale, Machine Learning (ML) și Deep Learning (DL). Importanța sa rezidă în definirea schimbării de paradigmă de la programarea clasică (AI Simbolic, bazată pe reguli explicite) la modelul modern de ML, în care sistemele **învață automat regulile** direct din date. Punctul central al lecției este înțelegerea modului în care Machine Learning realizează acest lucru, și anume prin **învățarea reprezentărilor utile ale datelor**. Înțelegerea ierarhiei AI și a celor patru ramuri de Machine Learning (Supervizată, Nesupervizată, Auto-supervizată și prin Întărire) este esențială pentru a aborda modelele liniare și alte tehnici avansate.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Structura Ierarhică:** Inteligența Artificială (AI) este domeniul cel mai larg, incluzând Machine Learning (ML), care la rândul său include Deep Learning (DL).
2. **Paradigma ML:** Machine Learning reprezintă o schimbare față de programarea clasică; în loc să creeze manual regulile, sistemul este **antrenat** să descopere regulile prelucrării datelor pe baza datelor de intrare și a rezultatelor așteptate.
3. **Mecanismul de Învățare:** Învățarea în ML se concentrează pe **transformarea semnificativă a datelor** (învățarea reprezentărilor) pentru a găsi o modalitate simplă de a rezolva problema (ex: clasificare simplă prin $x > 0$).
4. **Clasificarea Problemelor:** Majoritatea problemelor de ML se încadrează în **Clasificare** (etichetă discretă) sau **Regresie** (valoare continuă) și sunt abordate prin cele patru ramuri principale ale învățării (Supervizată, Nesupervizată, Auto-supervizată, prin Întărire).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

1.1 Inteligența Artificială, Machine Learning și Deep Learning

- Relația dintre cele trei concepte este prezentată ca fiind ierarhică și concentrică (Figura 1):
  - **Inteligența Artificială** (Artificial intelligence)
  - ⊃ **Machine Learning**
  - ⊃ **Deep Learning**

1.1.1 Inteligența Artificială

- AI reprezintă efortul de a **automatiza sarcini intelectuale** care sunt în mod normal îndeplinite de oameni.
- **AI Simbolic (Symbolic AI):** Programatorii creează manual un set suficient de mare de **reguli explicite** pentru manipularea cunoștințelor.
- Popularitate: AI Simbolic a atins apogeul în timpul **boom-ului sistemelor expert (expert systems) din anii 1980**.
- Aplicabilitate: Este potrivit pentru rezolvarea problemelor logice, bine definite.
- Limitare: Devine **intractabil** (prea dificil) să se găsească reguli explicite pentru rezolvarea problemelor mai complexe sau „fuzzy”.

1.1.2 Machine Learning (ML)

- ML propune o schimbare de paradigmă față de programarea clasică, răspunzând la întrebarea: Ar putea un computer să învețe automat aceste reguli **uitându-se la date**, în loc ca programatorii să le creeze manual?.
- Un sistem ML este **antrenat (trained)**, spre deosebire de a fi programat explicit.
- Schema Programării Clasice: Reguli + Date $\rightarrow$ Răspunsuri (Answers).
- Schema Machine Learning: Date + Răspunsuri $\rightarrow$ Reguli (Rules).
- Istoric: Deși ML a început să înflorească **abia în anii 1990**, a devenit rapid cel mai popular și de succes subdomeniu al AI.

1.1.3 Învățarea Reprezentărilor din Date (Learning representations from data)

- Machine Learning funcționează prin descoperirea regulilor necesare pentru a executa o sarcină de prelucrare a datelor, pe baza unor exemple de rezultate așteptate.
- Pentru a face Machine Learning, sunt necesare trei elemente:
  1. Puncte de date de intrare (*Input data points*).
  2. Exemple ale rezultatului așteptat (*Examples of the expected output*).
  3. O modalitate de a măsura cât de bine funcționează algoritmul (*A way to measure whether the algorithm is doing a good job*).

- **Problema Centrală:** Este transformarea **semnificativă** a datelor de intrare pentru a obține **reprezentări utile** care să ne apropie de rezultatul așteptat.
- **Reprezentarea:** Este un mod diferit de a privi datele – de a le reprezenta sau codifica.
- **Exemplu (Figura 4):** O problemă de clasificare care este complexă în reprezentarea inițială (1: Raw data) poate fi rezolvată simplu printr-o **schimbare de coordonate (Coordinate change)**, rezultând o reprezentare mai bună (3: Better representation).
    - Regula simplă de clasificare devine: "Punctele negre sunt cele pentru care **x > 0**", iar "Punctele albe sunt cele pentru care **x < 0**".
- **Învățarea (Learning):** În acest context, descrie un proces automat de căutare sistematică a diferitelor schimbări posibile de coordonate (reprezentări), utilizând feedback-ul (procentul de puncte clasificate corect).
- **Spațiul de Ipoteze (hypothesis space):** Algoritmii ML caută soluții doar printr-un set predefinit de operațiuni. Ei nu sunt creativi în găsirea transformărilor.
- **Două Tipuri Principale de Probleme ML**:
    1. **Clasificare (Classification):** Scopul este de a prezice o **singură etichetă discretă** pentru un punct de date de intrare.
    2. **Regresie (Regression):** Constă în prezicerea unei **valori continue** în loc de o etichetă discretă.

1.1.4 Patru Ramuri ale Machine Learning

1. **Învățare Supervizată (Supervised learning)**
    - **Obiectiv:** Învățarea mapării datelor de intrare la **ținte cunoscute** (numite și *adnotări - annotations*).
    - Aplicații: Reprezintă aproape toate aplicațiile ML de succes din zilele noastre (ex: recunoașterea vorbirii, clasificarea imaginilor, traducerea lingvistică).
    - Variante (bazate pe clasificare și regresie):
        - Generarea de Secvențe (Sequence generation).
        - Prezicerea Arborelui Sintactic (Syntax tree prediction).
        - Detecția Obiectelor (Object detection) (problemă combinată de clasificare și regresie vectorială).
        - Segmentarea Imaginii (Image segmentation).
2. **Învățare Nesupervizată (Unsupervised learning)**
    - **Obiectiv:** Găsirea de transformări interesante ale datelor de intrare **fără ajutorul vreunui rezultat țintă**.
    - Scopuri: vizualizarea datelor, compresia datelor, eliminarea zgomotului (*denoising*) sau înțelegerea corelațiilor.
    - Importanță: Este adesea un pas necesar pentru a înțelege mai bine un set de date înainte de a aborda o problemă supervizată.
    - Categorii Cheie: **Reducerea dimensionalității (dimensionality reduction)** și **Gruparea (clustering)**.
3. **Învățare Auto-supervizată (Self-supervised learning)**

- Este o instanță specifică de învățare supervizată, dar se diferențiază prin faptul că este **fără etichete adnotate de oameni**.
- Etichetele (țintele) sunt generate din datele de intrare în sine, de obicei cu un algoritm euristic.
- Exemplu Cheie: **Autoencoderele (autoencoders)**, unde ținta generată este intrarea (input-ul) neschimbată.
- Exemple Temporale: Prezicerea cadrului următor într-un videoclip sau a cuvântului următor dintr-un text (supravegherea vine din datele de intrare viitoare).

4. **Învățare prin Întărire (Reinforcement learning - RL)**
   - Istoric: A atras atenția recent, după ce Google DeepMind l-a aplicat cu succes pentru a juca jocuri Atari și Go.
   - **Mecanism:** Un **agent** primește informații despre mediul său și învață să aleagă acțiuni care vor **maximiza o anumită recompensă (reward)**.
   - Aplicații: În prezent, este în mare parte o zonă de cercetare cu succese practice limitate dincolo de jocuri.
   - Viitor: Se așteaptă să preia o gamă tot mai largă de aplicații din lumea reală: mașini autonome, robotică, managementul resurselor, educație.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Concept Dificil: AI Simbolic vs. ML (Schimbarea de Paradigmă)

- **Explicație Simplificată (Context):** Să ne imaginăm că încercăm să învățăm un robot să diferențieze o pisică de un câine.
  - **AI Simbolic (Anii '80):** Programatorul încearcă să scrie o listă lungă și detaliată de reguli ("Dacă are urechi ascuțite *și* mustăți lungi *și* face 'miau', atunci e pisică."). Dar ce se întâmplă dacă pisica e văzută din profil sau e un pui? Scrierea tuturor regulilor posibile (pentru a rezolva problema *fuzzy*) devine **intractabilă**.
  - **Machine Learning (Anii '90+):** Programatorul renunță să scrie regulile. În schimb, hrănește sistemul cu mii de imagini etichetate ("Aceasta e pisică", "Acesta e câine"). Sistemul **învață automat** regulile subiacente, fără să știe exact cum arată acele reguli.

Concept Dificil: Învățarea Reprezentărilor

- **Definiție:** A găsi o "reprezentare" înseamnă a transforma datele brute într-un format care scoate în evidență informația importantă.
- **Analogie (ELI5):** Gândiți-vă la datele brute (imaginea 1 din Figura 4) ca la o hartă unde granițele dintre țări sunt invizibile. Sarcina (clasificarea) este imposibilă. O **reprezentare bună** (imaginea 3) este ca și cum ați roti harta și ați desena o linie clară exact pe graniță

(axa Y în exemplu). Acum, clasificarea este trivială: tot ce e la stânga este o țară, tot ce e la dreapta este cealaltă. Mașina învață prin căutarea automată a celei mai bune rotații/transformări (schimbare de coordonate).

Concept Dificil: Spațiul de Ipoteze

- **Clarificare:** Algoritmii ML caută cel mai bun model, dar nu pot căuta *oriunde*. Dacă mașina este un simplu model liniar, **spațiul de ipoteze** este limitat la toate liniile drepte posibile. Dacă transformarea necesară ar fi un cerc, algoritmul nu ar găsi-o, deoarece cercurile nu fac parte din setul său predefinit de operațiuni. Asta se întâmplă deoarece algoritmii nu sunt creativi; ei doar efectuează căutări automate.

Termeni Tehnici Clasificați

- **Clasificare:** Răspunsul este o alegere dintr-un set limitat (discret). Ex: Este pisică sau câine?.
- **Regresie:** Răspunsul este o valoare numerică continuă. Ex: Cât va costa o casă? Cât de mare va fi cutia de delimitare (bounding box) în detecția obiectelor?.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

Relația Cauză-Efect: Declinul AI Simbolic și Ascensiunea ML

- **Cauză:** AI Simbolic (bazat pe reguli) a atins rapid limitele de **intractabilitate** atunci când a fost aplicat la probleme complexe sau insuficient definite ("fuzzy").
- **Efect:** Această limitare a forțat o schimbare de paradigmă. Machine Learning a apărut ca o soluție, inversând procesul: dacă nu putem scrie regulile, lăsăm mașina să le **învețe** din exemple.

Conexiunea dintre Reprezentare și Succesul ML

- **Conexiune:** Succesul oricărui algoritm de Machine Learning depinde direct de calitatea **reprezentării** datelor. Dacă datele sunt prost reprezentate, chiar și cel mai bun algoritm de clasificare va eșua, deoarece nu poate găsi o regulă simplă (ca $x > 0$) pentru a le separa.
- **Implicație:** Procesul de *învățare* în ML este, fundamental, procesul de **căutare automată a unei reprezentări mai bune**.

Conexiuni Între Ramurile ML

- **Învățarea Nesupervizată** este adesea o componentă de *preprocesare* pentru **Învățarea Supervizată**. De exemplu, reducerea dimensionalității (Nesupervizată) poate simplifica datele brute, îmbunătățind performanța și timpul de antrenare (Supervizată).

- **Învățarea Auto-supervizată** este o punte ingenioasă. Folosește logica strictă a **Învățării Supervizate** (necesitatea unei ținte/etichete), dar rezolvă problema costisitoare a etichetării umane prin generarea automată a acestor etichete din datele însele.

Metaforă pentru înțelegerea Învățării prin Întărire (RL):

Gândiți-vă la Învățarea prin Întărire ca la dresarea unui câine, fără a-i da instrucțiuni clare (spre deosebire de programarea clasică). Nu-i spui "stai jos", dar de fiecare dată când stă jos întâmplător, îi dai o recompensă (o bucățică de mâncare). Câinele (agentul) explorează mediul, face acțiuni și, pe măsură ce primește recompense, învață singur care sunt acțiunile optime pentru a-și **maximiza recompensa**.

1.2

Salutări! Mă bucur să vă ajut să aprofundați fundamentele Regresiei Liniară, un concept esențial în Machine Learning. Vom parcurge împreună materialul, asigurându-ne că fiecare detaliu matematic și conceptual este clarificat.

Ghid de Studiu Complet: 1.2 Regresia Liniară (Linear Regression)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Această lecție se concentrează pe **Regresia Liniară (Linear Regression)**, o metodă de **învățare supervizată** utilizată pe scară largă pentru **prezicerea unei ieșiri cu valoare reală** (regresie). Ea reprezintă cel mai simplu model de Machine Learning bazat pe ipoteza că există o **relație liniară** între variabilele de intrare (caracteristici) și variabila dependentă (ținta). Importanța acestei lecții este dublă: oferă un model fundamental și robust și, crucial, demonstrează modul în care optimizarea statistică (Principiul Maximum Likelihood) este echivalentă cu minimizarea unei funcții de cost binecunoscute în ML, **Eroarea Pătratică Medie (Mean Squared Error - MSE)**. Înțelegerea derivării analitice a soluției (Ecuația Normală) este vitală pentru a sesiza limitările computaționale ale modelelor liniare la scară mare.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Definiție și Obiectiv:** Regresia Liniară prezice o valoare continuă $y \in \mathbb{R}$ pe baza unei **sume ponderate liniar** a caracteristicilor de intrare $x_1, \ldots, x_d$, la care se adaugă un **termen de bias** w_0.
2. **Modelul Matematic:** Modelul este o **transformare afină** a caracteristicilor și poate fi exprimat compact sub formă vectorială/matriceală ca $\hat{y} = \mathbf{w}^\top \mathbf{x}$.
3. **Funcția de Cost:** Sub ipoteza că **zgomotul (noise)** urmează o distribuție normală, găsirea celor mai bune ponderi (Maximum Likelihood) este echivalentă cu minimizarea

Funcției de Pierdere Mean Squared Error (MSE) pe întregul set de antrenare $\mathcal{L}(\mathbf{w})$.

4. **Soluția Analitică:** Vectorul de ponderi optim \mathbf{w}^_ poate fi calculat direct, fără iterații, folosind **Ecuația Normală (Normal Equation)**: \mathbf{w}^_ = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

1.2 Regresia Liniară: Fundamente și Notații

- **Regresia Liniară** este o metodă larg utilizată pentru a prezice o ieșire cu valoare reală (variabilă dependentă $y \in \mathbb{R}$), numită **label** sau **target**, pe baza unui set de intrări cu valoare reală (variabile independente) numite **features** sau **covariates** $(x_1, x_2, \ldots, x_d)$.
- Se presupune că relația dintre y și variabilele independente este **liniară** (o sumă ponderată a caracteristicilor).
- Se presupune că **zgomotul (noise)** pe observații urmează o **distribuție normală (normal distribution)**.
- Pentru a dezvolta modelul este necesar un **dataset** numit **training dataset** sau **training set**, în care fiecare rând este un **example**, **data point** sau **sample**.

Formularea Modelului

- **Notații:**
  - n este numărul de exemple din setul de date.
  - d este numărul de caracteristici (*features*).
  - x_j este a j-a caracteristică.
  - $y^{(i)}$ este eticheta cunoscută (reală) pentru exemplul i.
  - w_j este a j-a pondere (*weight*), inclusiv **bias-ul** sau **termenul de interceptare** w_0.
- **Modelul Liniar în Forma Scalară (Ecuația 1):**

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d$$

  - $\hat{y}$ este valoarea prezisă (*predicted value*).
- **Transformarea Afină:** Strict vorbind, Ecuația (1) este o **transformare afină (*affine transformation*)** a caracteristicilor de intrare, care combină o transformare liniară (suma ponderată) cu o translație (bias-ul adăugat w_0).
- **Modelul Liniar în Forma Vectorială (Ecuația 2 și 3):**

$$\hat{y} = \mathbf{w}^\top \mathbf{x} \text{ sau } \hat{y} = \mathbf{x}^\top \mathbf{w}$$

  - $\mathbf{w}$ este **model's weight vector** (vectorul de ponderi), care include $w_0, w_1, \ldots, w_d$.
  - $\mathbf{x}$ este **instance's feature vector** (vectorul de caracteristici al instanței), care include $x_0 = 1$ (dummy feature) pentru a permite încorporarea bias-ului w_0 în produsul scalar.

- $\mathbf{w}^\top$ este transpusa lui $\mathbf{w}$ (un vector rând), iar $\mathbf{w}^\top\mathbf{x}$ reprezintă produsul scalar.
- **Modelul Matriceal pentru Întregul Set de Date (Ecuația 4):**

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$$

  - $\mathbf{X} \in \mathbb{R}^{n\times(d+1)}$ este **design matrix** (matricea de design).
  - $\mathbf{X}$ conține un rând pentru fiecare exemplu (n exemple) și o coloană pentru fiecare caracteristică plus coloana de 1s (pentru bias).
  - $\hat{\mathbf{y}} \in \mathbb{R}^n$ este vectorul de predicții pentru toate cele n exemple.

Derivarea Funcției de Cost (Maximum Likelihood)

- **Obiectiv:** Găsirea vectorului de ponderi $\mathbf{w}$ care, în așteptare, va prezice noile exemple cu cea mai mică eroare (*lowest error*).
- **Ipoteza de Zgomot:** Relația reală în lumea reală încorporează un termen de zgomot ϵ:

$$y = \mathbf{w}^\top\mathbf{x} + \epsilon, \text{ unde } \epsilon \sim \mathcal{N}(0, \sigma^2)$$

  - $\mathcal{N}(\mu, \sigma^2)$ este distribuția normală cu media μ și varianța σ^2.
- **Likelihood:** Aplicând Principiul Maximum Likelihood, se caută ponderile $\mathbf{w}$ care maximizează probabilitatea (likelihood-ul) de a vedea întregul set de date:

$$p(\mathbf{y}|\mathbf{X}, \mathbf{w}) = \prod_{i=1}^{n} p(y^{(i)}|\mathbf{x}^{(i)}, \mathbf{w})$$

  (Ecuația 5)
- **Echivalența cu MSE:** Pentru a simplifica optimizarea, se minimizează **log-likelihood-ul negativ (*negative log-likelihood*)**. Prin eliminarea termenilor constanți (care nu depind de $\mathbf{w}$) sub presupunerea că varianța σ este fixă, log-likelihood-ul negativ devine:

$$\sum_{i=1}^{n} \frac{1}{2}(y^{(i)} - \hat{y}^{(i)})^2$$

Funcția de Pierdere (Loss Function)

- **Măsura Calității:** Calitatea unui model este dată de **loss function** (funcția de pierdere), care cuantifică distanța dintre valoarea reală (y) și cea prezisă ($\hat{y}$). O pierdere mai mică este mai bună.
- **Squared Error:** Cea mai populară funcție de pierdere pentru probleme de regresie este **squared error** (eroarea pătratică).
  - Pierderea pentru un singur exemplu i: $l^{(i)}(\mathbf{w}) = \frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2$. (Constanta $\frac{1}{2}$ este adăugată din convenienței matematică pentru simplificarea derivatelor).
- **Funcția de Cost Globală (MSE - Mean Squared Error) (Ecuația 6):** Pentru a măsura calitatea modelului pe întregul set de date de n exemple, se calculează media

pierderilor:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n} l^{(i)}(\mathbf{w}) = \frac{1}{2n}(\mathbf{X}\mathbf{w} - \mathbf{y})^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$$

- **Obiectivul Antrenării:** Găsirea vectorului optim de ponderi \mathbf{w}^_ care **minimizează pierderea totală**:

\mathbf{w}^_ = \arg \min_{\mathbf{w}} \mathcal{L}(\mathbf{w})

Soluția Analitică (Ecuația Normală)

- Pentru a găsi minimul funcției de pierdere $\mathcal{L}(\mathbf{w})$, se calculează **gradientul** $\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w})$ și se egalează cu $\mathbf{0}^\top$.
- Acest proces conduce la **soluția analitică (closed-form solution)**, cunoscută sub numele de **Ecuația Normală (normal equation)**:

$$\mathbf{w}^* = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$$

- **Pseudo-inversa:** Matricea $\mathbf{X}^\dagger := (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$ este numită **pseudo-inversa (pseudo-inverse)** sau **inversa Moore-Penrose** a matricei $\mathbf{X}$.
- **Calculul Pseudo-inversei:** Este adesea calculată folosind tehnica de factorizare a matricii numită **descompunerea valorii singulare (SVD - singular value decomposition)**: $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$.
  - Abordarea SVD este mai eficientă în calcul decât Ecuația Normală directă și poate gestiona cazurile în care $\mathbf{X}^\top\mathbf{X}$ nu este inversabilă (singulară), cum ar fi când $n < d$ (mai puține exemple decât caracteristici) sau când caracteristicile sunt redundante.

Complexitatea Computațională

- **Antrenarea (Găsirea $\mathbf{w}^*$):**
  - Ecuația Normală implică inversarea unei matrici $(d+1) \times (d+1)$. Complexitatea este tipic $O(d^{2.4})$ sau $O(d^3)$, în funcție de implementare.
  - Abordarea SVD are o complexitate de aproximativ $O(d^2)$ (în anumite contexte, deși soluția completă implică n).
  - Ambele abordări devin foarte lente când **numărul de caracteristici d devine mare (e.g., 100,000)**.
  - Pe partea pozitivă, ambele abordări sunt liniare în raport cu numărul de instanțe n din setul de antrenare ($O(n)$).
- **Predicția (Utilizarea $\mathbf{w}^*$):** Odată ce modelul este antrenat, predicțiile sunt **foarte rapide**, complexitatea computațională fiind liniară în raport cu numărul de instanțe și numărul de caracteristici.

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Abstract: Transformarea Afină (Affine Transformation)

- **Explicație Simplificată:** În termeni de geometrie, o transformare liniară pură (ex: w_1x_1) aplică doar scalări și rotații; ea trebuie să treacă prin origine $(0, 0)$. O **transformare afină** (modelul nostru $\hat{y} = w_0 + \ldots$) este o transformare liniară **plus o translație**.
- **Rolul Bias-ului (w_0):** Bias-ul w_0 este termenul de translație. El indică valoarea prezisă $\hat{y}$ atunci când **toate caracteristicile x_i sunt egale cu zero**. Acest lucru permite dreptei (sau planului) să se deplaseze liber în spațiu, oferind modelului o flexibilitate mult mai mare de a se potrivi datelor.

Concept Dificil: Echivalența MSE și Maximum Likelihood (ML)

- **Context:** De ce folosim eroarea pătratică? Aceasta nu este o alegere arbitrară.
- **Asta se întâmplă deoarece:** Prin presupunerea că erorile de măsurare (zgomotul ϵ) sunt distribuite normal (curbă Gaussiană), optimizarea statistică (găsirea ponderilor care fac datele observate cel mai probabil - Maximum Likelihood) **matematic se reduce** la minimizarea sumei diferențelor la pătrat. MSE este o consecință directă și elegantă a unei ipoteze statistice standard.

Termen Tehnic: Matricea de Design $\mathbf{X}$ (Design Matrix)

- **Analogie (Gândiți-vă la Excel):** Dacă aveți o foaie de calcul cu datele dvs., **matricea de design $\mathbf{X}$** este, în esență, toate coloanele de intrare (caracteristicile). Dar există o mică "păcăleală": trebuie să adăugați o coloană suplimentară, formată doar din numărul 1, la începutul foii.
- *De ce?* Această coloană de 1s este necesară pentru a încorpora **bias-ul** w_0 în înmulțirea matricială $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$. Dacă bias-ul w_0 se înmulțește cu 1, el rămâne o translație constantă.

Concept Computațional: Limitarea Complexității $O(d^3)$

- **Explicație Simplificată:** Deși Ecuația Normală oferă o soluție exactă, ea implică inversarea unei matrici de dimensiune $(d + 1) \times (d + 1)$.
- **Analogie:** Gândiți-vă la d ca la dimensiunea unei cutii. Timpul necesar pentru a procesa acea cutie crește de trei ori mai repede decât dimensiunea ei (puterea a 3-a). Dacă d este 10, timpul e 1.000. Dacă d e 100, timpul e 1.000.000. Dacă d este 100.000 (ceea ce se întâmplă adesea în ML modern), metoda devine exponențial de lentă.
- **Implicație:** De aceea, metodele care depind de d la o putere mare ($O(d^3)$) sunt nepotrivite pentru seturile de date cu număr mare de caracteristici, chiar dacă sunt rapide pentru un număr mare de instanțe ($O(n)$).

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația dintre Erroare și Pierdere (Contribuția Pătratică):** Faptul că Regresia Liniară folosește **eroarea pătratică** (MSE) implică o relație cauză-efect: **diferențele mari** între valoarea reală și cea prezisă (outliers) duc la **contribuții mult mai mari** la funcția de pierdere totală, datorită naturii pătratice a erorii. Acest lucru face ca modelul să fie sensibil la valorile extreme și să încerce să le "potrivească" cât mai bine, minimizând erorile mari.
- **Conexiunea SVD și Ecuația Normală:** Deși Ecuația Normală oferă soluția teoretică $\mathbf{w}^* = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$, în practică, calculul direct al inversei $(\mathbf{X}^\top\mathbf{X})^{-1}$ este adesea evitat.
  - **Cauză:** Calculul inversei poate fi instabil numeric sau imposibil (dacă matricea este singulară).
  - **Efect:** De aceea, se preferă calcularea **pseudo-inversei** (Moore-Penrose inverse) folosind **SVD**. SVD este mai robust, mai eficient și garantează o soluție chiar și în cazurile dificile ($n < d$ sau caracteristici redundante).
- **Implicațiile în Fluxul de Lucru ML:** Înțelegerea complexității computaționale $O(d^3)$ ne pregătește pentru lecțiile viitoare. Deși LR oferă o soluție exactă (analitică), limitarea sa la d mare ne forțează să căutăm **o altă modalitate de antrenare** a aceluiași model liniar, care să fie mai potrivită pentru seturi de date extinse (așa cum este menționat la finalul materialului furnizat).

---

5. EXERCIȚII ȘI REZOLVARE

Exercițiul 1: Identificarea Componentelor Modelului

Gândiți-vă la o problemă de regresie liniară simplă în care încercați să preziceți prețul unei case.

Identificați în formularea de mai jos rolul fiecărui termen, bazându-vă pe Ecuația (1):

$$\text{Preț Prezis} = 50.000 + 150 \times (\text{Suprafață}) - 10.000 \times (\text{Vechime})$$

1. Care este valoarea bias-ului (w_0)?
2. Câte caracteristici (d) are modelul?
3. Care este ponderea asociată cu caracteristica "Suprafață"?
4. Dacă toate caracteristicile ar fi zero, care ar fi prețul prezis?

Rezolvare 1:

1. Bias-ul (w_0) este **50.000**.
2. Modelul are **două** caracteristici ($d = 2$): Suprafața și Vechimea.
3. Ponderea (w_1) asociată cu Suprafața este **150**.
4. Prețul prezis ar fi egal cu bias-ul, adică **50.000**.

Exercițiul 2: Calculul Predicției (Forma Scalară)

Folosind modelul de la Exercițiul 1:

$$\hat{y} = 50.000 + 150 \cdot x_{\text{suprafață}} - 10.000 \cdot x_{\text{vechime}}$$

Calculați prețul prezis ($\hat{y}$) pentru o casă cu următoarele caracteristici:

- Suprafață ($x_{\text{suprafață}}$) = 120 mp
- Vechime (x_{vechime}) = 5 ani

Rezolvare 2:

Se substituie valorile în ecuație:

$$\hat{y} = 50.000 + 150 \times 120 - 10.000 \times 5$$

1. $150 \times 120 = 18.000$
2. $10.000 \times 5 = 50.000$

$$\hat{y} = 50.000 + 18.000 - 50.000$$

$$\hat{y} = 18.000$$

Prețul prezis este **18.000** (Unitatea de măsură nu este specificată în surse, dar logic ar fi USD/EUR/unități monetare).

Exercițiul 3: Relația dintre MSE și Log-Likelihood Negativ

În Regresia Liniară, care este legătura fundamentală (în termeni de obiectiv matematic) dintre funcția de pierdere **Mean Squared Error (MSE)** și optimizarea bazată pe probabilitate **Negative Log-Likelihood**?

Rezolvare 3:

Conform surselor, obiectivul de a minimiza **Negative Log-Likelihood** (sub ipoteza de zgomot cu distribuție normală) este **egal** (până la constante multiplicative) cu minimizarea funcției de pierdere **Mean Squared Error (MSE)**. Acest lucru înseamnă că, deși provin din domenii diferite (statistică versus optimizare ML), ele ne conduc la **aceeași soluție optimă** pentru vectorul de ponderi $\mathbf{w}^*$.

1.3

Bine ați venit! Mă bucur să vă ghidez prin Algoritmul de Coborâre a Gradientului (*Gradient Descent*), o metodă fundamentală și universală de optimizare utilizată în Machine Learning. În timp ce Regresia Liniară are o soluție analitică (Ecuația Normală), Coborârea Gradientului devine absolut necesară pentru seturi de date mari și pentru modele mai complexe, cum ar fi rețelele neuronale adânci.

Ghid de Studiu Complet: 1.3 Coborârea Gradientului (Gradient Descent)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Această lecție introduce **Coborârea Gradientului (Gradient Descent - GD)**, un algoritm generic de optimizare iterativă, capabil să găsească soluții optime pentru o gamă largă de probleme. Rolul său esențial în Machine Learning este de a **minimiza funcția de pierdere (loss function)** (în cazul Regresiei Liniară, MSE) prin ajustarea repetată a parametrilor (vectorul de ponderi $\mathbf{w}$). Spre deosebire de soluția analitică, GD se scalează bine cu numărul de caracteristici, devenind astfel metoda de bază pentru antrenarea modelelor complexe, chiar dacă antrenarea este un proces mai lent și mai puțin exact decât soluția analitică (în cazul Regresiei Liniară). Lecția explorează, de asemenea, cele trei variante principale ale algoritmului: Batch, Stochastic și Mini-Batch.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Mecanismul de Optimizare:** Coborârea Gradientului este un algoritm iterativ care ajustează parametrii (ponderile $\mathbf{w}$) urmând direcția de **cea mai abruptă pantă descendentă**, dată de **vectorul gradientului negativ**.
2. **Formula de Actualizare:** Parametrii sunt actualizați la fiecare pas prin formula $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$, unde η este rata de învățare (*learning rate*).
3. **Rata de Învățare (η):** Este un **hiperparametru** care determină mărimea pasului. O rată prea mică încetinește convergența, iar una prea mare poate duce la **divergență** (eșecul de a găsi soluția).
4. **Cele Trei Variante (BGD, SGD, MBGD):** Diferența principală constă în **câte exemple sunt folosite** pentru a calcula gradientul la fiecare pas, afectând viteza, regularitatea și capacitatea de a scăpa de minimele locale.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

1.3 Coborârea Gradientului (Gradient Descent)

- **Definiție:** Un algoritm generic de optimizare capabil să găsească soluții optime la o gamă largă de probleme.
- **Obiectiv:** Actualizează parametrii **iterativ** pentru a minimiza funcția de pierdere.
- **Principiul de Funcționare:** Măsoară gradientul local al funcției de pierdere $\mathcal{L}$ în raport cu vectorul de parametri $\mathbf{w}$ și se deplasează în direcția de **cea mai abruptă coborâre**, dată de **vectorul gradientului negativ**.
- **Convergența:** Un minim al funcției de pierdere a fost atins când **gradientul este zero**.

Pașii Algoritmului și Formula de Actualizare

1. **Inițializarea Aleatorie (*random initialization*):** Valorile ponderilor modelului sunt inițializate, de obicei aleatoriu.
2. **Actualizarea Iterativă:** Parametrii sunt actualizați pas cu pas, în direcția gradientului negativ, până când algoritmul converge la un minim.
3. **Formula de Actualizare (Matematică):**

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \left(\frac{1}{n} \sum_{i=1}^{n} l^{(i)}(\mathbf{w}) \right)$$

4. **Gradientul pentru Regresia Liniară (Ecuația 8):**

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \frac{1}{n} \mathbf{X}^{\top} (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Hiperparametrii și Rata de Învățare (η)

- **Hiperparametrii:** Parametri care sunt **reglați manual** (tunable), dar nu sunt actualizați în bucla de antrenare.
- **Rata de Învățare (η):** Un hiperparametru important care determină **mărimea pașilor**.
  - η **prea mic:** Algoritmul necesită multe iterații pentru a converge și va dura mult timp.
  - η **prea mare:** Algoritmul poate sări peste minim (să sară valea) și poate **diverge**, eșuând în găsirea unei soluții bune.

Tipuri de Coborâre a Gradientului

Tip	Denumire	Calculul Gradientului	Caracteristici Cheie
1.	**Batch Gradient Descent (BGD)**	Folosește **întregul set de antrenare** $\mathbf{X}$ la fiecare pas.	Foarte lent pe seturi de date foarte mari. Converge precis la minim. Se scalează bine cu numărul de caracteristici (d).
2.	**Stochastic Gradient Descent (SGD)**	Alege o **instanță aleatorie** (i) și calculează gradientul bazat doar pe acea instanță.	**Foarte rapid**. Permite antrenarea pe seturi de date uriașe, deoarece necesită puțină memorie per iterație. Din cauza naturii aleatorii, progresul este neregulat (bounce up and down) și **nu se așază niciodată exact la minim**. Formula de actualizare este $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} l^{(i)}(\mathbf{w})$.
3.	**Mini-Batch Gradient Descent (MBGD)**	Calculează gradientul pe un **set mic aleatoriu de instanțe** numit *mini-batch* $\mathcal{B}$.	Este mai rapid decât BGD și progresul este **mai puțin neregulat** decât SGD. Dimensiunea mini-batch-ului ($

- **Epocă (*epoch*):** Un ciclu de n iterații (unde n este numărul de exemple), în care fiecare exemplu este utilizat pentru a actualiza ponderile modelului. Setul de antrenare trebuie amestecat (*shuffle*) pentru a asigura că algoritmul parcurge fiecare instanță.

Provocări și Proprietăți ale Funcției de Pierdere

- **Funcții de Pierdere Neregulate:** Pot avea gropi, creste, platouri (*plateaus*) care fac convergența dificilă.
- **Minim Local vs. Global:** Algoritmul poate converge la un **minim local** (local minimum), care nu este la fel de bun ca **minimul global** (global minimum).
- **Vantajul Regresiei Liniară:** Funcția de pierdere MSE este o **funcție convexă (*convex function*)**.
  - Aceasta înseamnă că există **un singur minim global** și nu există minime locale.
  - Drept consecință, Coborârea Gradientului este garantată să se apropie de minimul global (dacă η nu este prea mare).
- **Scalarea Caracteristicilor (*Feature Scaling*):** Dacă caracteristicile au scări (mărimi) foarte diferite, funcția de pierdere arată ca un *bol alungit*.
  - În acest caz, algoritmul GD se mișcă aproape ortogonal la direcția minimului, iar convergența va dura mult timp.
  - Soluție: Toate caracteristicile trebuie să aibă o scară similară.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Abstract: Gradientul Negativ

- **Explicație Simplificată (Analogie "Ceața din Munți"):** Imaginați-vă că sunteți pe un munte acoperit de o ceață densă și trebuie să ajungeți în cel mai jos punct al văii (minimul pierderii).
  - **Gradientul** este ca și cum ați măsura panta sub tălpile picioarelor. Gradientul arată întotdeauna **în sus** (direcția de creștere).
  - Pentru a coborî rapid, trebuie să mergeți în direcția **opusă** pantei (direcția gradientului negativ).

Concept Dificil: Convexitatea Funcției de Pierdere (MSE)

- **Definiție Simplificată:** O funcție este **convexă** dacă, oricare ar fi două puncte pe curbă, segmentul de linie care le unește nu trece niciodată pe sub curbă.
- **Asta se întâmplă deoarece:** Pentru Regresia Liniară (folosind MSE), suprafața de pierdere este o parabolă (sau un bol în dimensiuni superioare). Deoarece bolul are o formă perfectă, indiferent de unde începeți, există o singură gaură (minim) spre care veți

coborî. Această proprietate **garantează** că GD va găsi soluția globală (spre deosebire de modelele mai complexe, care nu sunt convexe și au minime locale).

Termen Tehnic: Hiperparametru vs. Parametru

- **Parametri (Greutăți w):** Sunt **învățați** automat de algoritm în timpul antrenării (se ajustează la fiecare pas de GD).
- **Hiperparametri (Rata de Învățare η):** Sunt **aleși manual** de către programator *înainte* de antrenare și rămân ficși pe parcursul rulării (sau sunt ajustați printr-un *learning schedule*). Reglarea lor se numește *hyperparameter tuning* și se face pe un *validation dataset*.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Metoda de Optimizare):** Deși Ecuația Normală oferă o soluție exactă, complexitatea sa computațională $O(d^3)$ o face nepractică pentru un număr mare de caracteristici (d). **Cauza:** Nevoia de a inversa o matrice mare. **Efect:** Adoptarea Coborârii Gradientului, care, deși produce o soluție aproximativă și necesită mai mult timp total (datorită iterațiilor), **scalează mai bine** cu d, fiind mult mai rapidă când d este în sute de mii.
- **Relația dintre Rata de Învățare și Convergență:** Rata de învățare η controlează stabilitatea și viteza algoritmului. Dacă η este aleasă greșit, algoritmul fie va rula inutil de mult (prea mic), fie va sări peste minim și va eșua (prea mare), afectând direct calitatea rezultatului final.
- **Compromisul Viteză vs. Precizie (SGD vs. BGD):**
  - BGD este lent, dar converge precis la minim (în cazul convex).
  - SGD este foarte rapid per pas, dar **stocastic** (aleatoriu) și, prin urmare, nu se așază niciodată exact la minim, ci "zboară" în jurul lui.
  - MBGD oferă un **compromis** bun, fiind mai stabil decât SGD, dar mult mai rapid decât BGD.
- **Generalizarea și Predicția:** Odată ce vectorul de ponderi $\hat{\mathbf{w}}$ este estimat (după antrenare), el este folosit pentru a estima ținta unui nou eșantion, proces numit **predicție** sau **inferență**.

---

5. EXERCIȚII ȘI REZOLVARE

Următorul exercițiu este preluat din sursă și ilustrează aplicarea soluției analitice (Ecuația Normală) pentru a calcula ponderile optime $\mathbf{w}^*$ într-un model simplu de regresie liniară.

Exercițiul 1: Calculul Ponderilor cu Ecuația Normală

Presupunem că avem un set de date format din trei puncte în planul 2D:

$$\text{Dataset } = (x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), (x^{(3)}, y^{(3)}) = (1, 2), (2, 4), (3, 5)$$

Vrem să estimăm următorul model de regresie liniară:

$$\hat{y} = w_1 x + w_0$$

Calculați:

1. Ponderile optime w_0 și w_1.
2. Predicțiile $\hat{y}^{(1)}, \hat{y}^{(2)}, \hat{y}^{(3)}$ corespunzătoare.

Soluție:

În acest caz, avem $n = 3$ exemple și $d = 1$ caracteristică. Vom folosi Ecuația Normală: $\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.

Pasul 1: Construirea Matricelor $\mathbf{X}$ și $\mathbf{y}$

Matricea de design $\mathbf{X}$ include o coloană de 1s pentru bias (w_0):

$$\mathbf{X} = \begin{bmatrix} 1 & x^{(1)} & 1 & x^{(2)} & 1 & x^{(3)} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 2 & 1 & 3 \end{bmatrix} \in \mathbb{R}^{3 \times 2}$$

Vectorul țintă $\mathbf{y}$ este:

$$\mathbf{y} = \begin{bmatrix} y^{(1)} & y^{(2)} & y^{(3)} \end{bmatrix} = \begin{bmatrix} 2 & 4 & 5 \end{bmatrix} \in \mathbb{R}^{3 \times 1}$$

Pasul 2: Calculul $\mathbf{X}^\top \mathbf{X}$ și $\mathbf{X}^\top \mathbf{y}$

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 2 & 1 & 3 \end{bmatrix} = \begin{bmatrix} 3 & 6 & 6 & 14 \end{bmatrix}$$

$$\mathbf{X}^\top \mathbf{y} = \begin{bmatrix} 1 & 1 & 1 & 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 2 & 4 & 5 \end{bmatrix} = \begin{bmatrix} 11 & 25 \end{bmatrix}$$

Pasul 3: Calculul $\mathbf{w}^*$ (Vectorul de Ponderi Optime)

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Inversa matricei $\mathbf{X}^\top \mathbf{X}$ este:

$$(\mathbf{X}^\top \mathbf{X})^{-1} = \begin{bmatrix} 7/3 & -1 & -1 & 1/2 \end{bmatrix}$$

Acum înmulțim:

$$\mathbf{w}^* = \begin{bmatrix} 7/3 & -1 & -1 & 1/2 \end{bmatrix} \begin{bmatrix} 11 & 25 \end{bmatrix} = \begin{bmatrix} (7/3 \times 11) - 25 & (-1 \times 11) + (1/2 \times 25) \end{bmatrix} = \begin{bmatrix} 77/3 - 75/3 & -22/$$

Răspunsul la punctul 1: Vectorul de ponderi optim este $\mathbf{w}^* = \begin{bmatrix} w_0 & w_1 \end{bmatrix} = \begin{bmatrix} 2/3 & 3/2 \end{bmatrix}$.

Deci, $w_0 = 2/3$ și $w_1 = 3/2$.

Pasul 4: Calculul Predicțiilor $\hat{\mathbf{y}}$

Calculăm $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$:

$$\hat{\mathbf{y}} = \begin{bmatrix} 1 & 1 & 1 & 2 & 1 & 3 \end{bmatrix} \begin{bmatrix} 2/3 & 3/2 \end{bmatrix} = \begin{bmatrix} 2/3 + 3/2 & 2/3 + 6/2 & 2/3 + 9/2 \end{bmatrix} = \begin{bmatrix} 4/6 + 9/6 & 4/6 + 18/6 & 4/6 + 27/$$

Răspunsul la punctul 2: Predicțiile sunt:

- $\hat{y}^{(1)} = 13/6$
- $\hat{y}^{(2)} = 22/6$
- $\hat{y}^{(3)} = 31/6$

1.4.

Înțeleg perfect! Trecem la un subiect esențial care ne arată cum putem folosi puterea regresiei liniare chiar și pentru a modela relații complexe, neliniare. Este o lecție critică pentru înțelegerea compromisului fundamental din Machine Learning: echilibrul dintre simplitate și complexitate.

Ghid de Studiu Complet: 1.4 Regresia Polinomială (Polynomial Regression)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Această lecție abordează limitarea majoră a Regresiei Liniară simple (incapacitatea de a se potrivi datelor neliniare) și introduce **Regresia Polinomială (Polynomial Regression)** ca soluție elegantă. Surprinzător, putem folosi un model formal **liniar** pentru a se potrivi datelor **neliniare**. Secretul constă în aplicarea unei **transformări neliniare** $\phi(\mathbf{x})$ asupra caracteristicilor de intrare, adăugând puteri și combinații ale acestora, dar păstrând modelul liniar **în parametri**. Importanța crucială a acestei secțiuni este că ne introduce în dinamica dintre complexitatea modelului (gradul M) și performanța de **generalizare** pe date nevăzute, conducând la fenomenul de **overfitting** (supra-antrenare) și la conceptul teoretic al **bias/variance trade-off**.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Principiul Fondator:** Regresia liniară se referă la faptul că modelul este **liniar în parametri** ($\mathbf{w}$), nu neapărat în caracteristici.
2. **Mecanismul de Extensie:** Regresia Polinomială extinde modelul liniar prin adăugarea **puterilor** (ex: x^2, x^3) și a **combinațiilor** (ex: a^2b) caracteristicilor inițiale ca noi caracteristici transformate $\phi(\mathbf{x})$.
3. **Evaluarea Performanței:** Calitatea modelului este evaluată folosind **Root Mean Square Error (RMSE)**, care este preferat deoarece are aceeași scală și unități ca și valorile țintă.
4. **Overfitting (Supra-antrenare):** Modelele cu un grad polinomial M prea mare se potrivesc perfect (sau aproape perfect) datelor de antrenare (eroare de antrenare mică), dar **oscilează sălbatic** și au o performanță slabă pe datele noi (eroare de testare mare), suferind de *overfitting*.

5. **Bias/Variance Trade-Off:** Eroarea de generalizare este suma erorilor de Bias, Varianță și Eroare Iredictibilă. Creșterea complexității (gradul M) reduce bias-ul, dar crește varianța, iar obiectivul este găsirea punctului optim (ex: $M = 4$) care minimizează eroarea de testare totală.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

1.4 Regresia Polinomială

- **Necesitatea:** Regresia Liniară clasică (care produce doar linii drepte) nu este suficient de expresivă pentru a se potrivi datelor mai complexe și mai interesante.
- **Soluția:** Putem adapta funcții neliniare în cadrul regresiei liniară, deoarece expresia "regresie liniară" se referă doar la faptul că modelul este **liniar în parametri**.
- **Transformarea Caracteristicilor ($\phi(\mathbf{x})$):** Se aplică o **transformare neliniară arbitrară** $\phi(\mathbf{x})$ asupra vectorului de intrare $\mathbf{x}$, iar componentele acestei transformări sunt combinate liniar.
- **Modelul Liniar Corespunzător (Ecuația 9):**

$$\hat{y} = \mathbf{w}^\top \phi(\mathbf{x})$$

  - $\phi : \mathbb{R}^d \to \mathbb{R}^D$ este transformarea neliniară.
  - **Vectorul de ponderi** $\mathbf{w}$ (care se află în $\mathbb{R}^M$) apare în continuare doar liniar în ecuație.

Generarea Caracteristicilor Polinomiale

- Cea mai simplă metodă de a realiza această transformare este prin **adăugarea puterilor** fiecărei caracteristici ca noi caracteristici.
- **Structura Vectorului** $\phi(\mathbf{x})$ pentru o singură caracteristică x:

$$\phi(x) = \left[1 \; x \; x^2 \vdots x^{M-1}\right] \in \mathbb{R}^M$$

- **Caracteristici Multiple:** Dacă există mai multe caracteristici (ex: a și b), Regresia Polinomială poate găsi **relații între ele**.
  - Aceasta se realizează prin adăugarea **tuturor combinațiilor de caracteristici** până la gradul dat.
  - Exemplu: Pentru gradul 3, se adaugă a^2, a^3, b^2, b^3, dar și combinațiile ab, a^2b, ab^2.

Măsura de Performanță: RMSE

- Calitatea modelului este evaluată prin pierderea $\mathcal{L}$.

- În loc de MSE, se utilizează frecvent **Root Mean Square Error (RMSE)**:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}^{(i)} - y^{(i)})^2} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\mathbf{w}^\top\phi(\mathbf{x}^{(i)}) - y^{(i)})^2}$$

- RMSE este utilă deoarece:
  - Permite compararea seturilor de date de dimensiuni diferite.
  - Are **aceeași scală și unități** ca valorile funcției observate $y^{(i)}$.

Alegerea Gradului M și Riscul de Overfitting

- **Alegerea M:** Cel mai bun grad M este cel care minimizează RMSE, găsit de obicei printr-o căutare *brute-force* a valorilor rezonabile.
- Pentru un set de antrenare de mărime n, este suficient să se testeze $0 \leq M \leq n-1$.
- **Cazuri Matematice (pentru n puncte de date):**
  - Dacă $M < n$, Estimatorul Maximum Likelihood (MLE) este unic.
  - Dacă $M \geq n$, există mai mulți parametri decât puncte de date. Se obține un sistem de ecuații liniare subdeterminat, rezultând **infinit de mulți estimatori MLE posibili**.
- **Demonstrarea Overfitting-ului (Exemplu cu $n = 10$):**
  - Polinoamele de grad mic ($M = 0, M = 1$) se potrivesc slab datelor (poor representations).
  - Pentru $M \in$, potrivirile sunt plauzibile.
  - Când gradul crește, se potrivește tot mai bine datelor de antrenare.
  - În cazul extrem $M = n - 1 = 9$, funcția **trece prin fiecare punct** de date.
  - Aceste polinoame de grad înalt **oscilează sălbatic**, sunt o reprezentare slabă a funcției subiacente și suferă de **overfitting**.
- **Eroarea de Testare (Generalizare):** Scopul este obținerea unei **generalizări bune** pe date noi (*unseen data*).
  - Eroarea de antrenare (*training error*, curba albastră) **nu crește niciodată** pe măsură ce gradul M crește.
  - Eroarea de testare (*test error*, curba portocalie) scade inițial, dar **crește semnificativ de la gradul 6 în sus**.
  - În exemplul dat, cea mai bună generalizare (cea mai mică eroare de testare) este obținută pentru **polinomul de grad $M = 4$**.

Bias/Variance Trade-off

- Eroarea de generalizare poate fi exprimată ca suma a trei erori distincte:
  1. **Bias (Părtinire):** Eroare datorată **presupunerilor greșite** despre model (ex: presupunem liniaritate când e cuadratic). Un Bias mare duce la **underfitting** (potrivire slabă).

2. **Varianță (Variance):** Eroare datorată **sensibilității excesive** a modelului la mici variații din datele de antrenare. O Varianță mare duce la **overfitting**.
3. **Eroare Iredictibilă (Irreducible error):** Eroare cauzată de zgomotul intrinsec al datelor (nu poate fi redusă decât prin curățarea surselor de date).

- **Compromisul:**
  - Creșterea complexității modelului (creșterea M) $\rightarrow$ **Crește varianța** și **Reduce bias-ul**.
  - Reducerea complexității modelului (scăderea M) $\rightarrow$ **Crește bias-ul** și **Reduce varianța**.
- Acest echilibru poartă numele de **bias/variance trade-off**.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Concept Dificil: Liniar în Parametri (Linear in the Parameters)

- **Analogie Simplificată (ELI5):** Gândiți-vă la o mașină (modelul) și la combustibil (parametrii $\mathbf{w}$).
  - În regresia liniară simplă, motorul arde combustibilul liniar: $y = w_1 \cdot x$.
  - În regresia polinomială, noi modificăm *combustibilul* înainte de a-l introduce, transformând x în x^2. Dar mașina folosește tot același tip de motor liniar, aplicând pur și simplu o pondere w_2 la acest combustibil nou: $y = w_1 \cdot x + w_2 \cdot x^2$.
  - Deoarece optimizarea matematică (Ecuația Normală sau GD) funcționează doar pe ponderile w_i, atâta timp cât w_i nu este ridicat la putere sau nu se află într-o funcție neliniară, modelul este considerat **liniar în parametri** și poate fi rezolvat ca atare.

Termen Tehnic: Overfitting

- **Analogie (Harta Detaliată):** Imaginați-vă că antrenați modelul să recunoască un traseu pe o hartă veche (datele de antrenare), care are pete de cerneală (zgomot) pe ea.
  - Un model supra-antrenat (grad M mare) nu doar că învață traseul, dar învață și **toate petele de cerneală**.
  - Când i se dă o hartă nouă, curată (date de testare), el caută petele de cerneală care nu există, deviază sălbatic (oscilează) și eșuează în a găsi traseul real.
  - *Asta se întâmplă deoarece:* Modelul a devenit excesiv de sensibil la zgomotul aleatoriu din setul de antrenare, o caracteristică a **varianței ridicate**.

MSE vs. RMSE

- **Context:** Deși minimizarea MSE este echivalentă cu minimizarea log-likelihood-ului negativ (așa cum am discutat anterior), **RMSE** este preferat pentru raportare și comparare.

- *Asta se întâmplă deoarece:* Prin aplicarea rădăcinii pătrate, RMSE are **aceeași scală și aceleași unități** ca și variabila țintă y, făcând eroarea mai ușor de interpretat în contextul lumii reale.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Legătura cu Regresia Liniară și Optimizarea:** Regresia Polinomială este o dovadă a faptului că puterea regresiei liniare nu stă doar în capacitatea sa de a modela drepte, ci în capacitatea sa de a fi antrenată eficient (analitic sau iterativ) atâta timp cât este liniară în $\mathbf{w}$. Prin transformarea $\phi(\mathbf{x})$, transformăm o problemă neliniară a datelor într-o problemă liniară într-un spațiu de dimensiuni superioare.
- **Relația Cauză-Efect (Bias/Variance):**
  - **CAUZĂ:** Folosirea unui grad M foarte mic (model simplu, ex: $M = 1$).
  - **EFECT:** Modelul nu este suficient de flexibil pentru a captura relația reală (e o presupunere greșită), rezultând un **Bias ridicat** și **Underfitting**.
  - **CAUZĂ:** Folosirea unui grad M foarte mare (model complex, ex: $M = 9$ pentru $n = 10$).
  - **EFECT:** Modelul este prea sensibil la date, rezultând o **Varianță ridicată** și **Overfitting**.
  - **Implicația Cheie:** Găsirea gradului optim M (ca $M = 4$ în exemplu) înseamnă găsirea **celui mai bun compromis** între Bias și Varianță.

---

5. EXERCIȚII ȘI REZOLVARE

Exercițiul 1: Expansiunea Polinomială a Caracteristicilor

Presupunem că avem un set de date cu două caracteristici, a și b, și dorim să aplicăm Regresia Polinomială de **gradul** $M = 2$.

1. Enumerați toate noile caracteristici care trebuie incluse în vectorul transformat $\phi(\mathbf{x})$, inclusiv termenul de bias ($x_0 = 1$).
2. Dacă un model de regresie liniară simplă cu a și b folosea 3 parametri ($\mathbf{w} \in \mathbb{R}^3$), câți parametri va folosi modelul de Regresie Polinomială de grad 2?

Rezolvare 1:

Pasul 1: Listarea caracteristicilor pentru gradul $M = 2$

Regula impune includerea tuturor puterilor și combinațiilor până la gradul 2.

- **Grad 0 (Bias):** 1
- **Grad 1:** a, b

- **Grad 2 (Puteri):** a^2, b^2
- **Grad 2 (Combinații):** ab

Vectorul de caracteristici transformat (pentru $\mathbf{x} = [a, b]$) este:

$$\phi(\mathbf{x}) = [1\ a\ b\ a^2\ b^2\ ab]$$

Pasul 2: Determinarea numărului de parametri

Numărul de parametri M este egal cu numărul de componente din vectorul $\phi(\mathbf{x})$. Modelul de Regresie Polinomială de grad 2 va folosi **6 parametri** (ponderi w_i), corespunzători fiecărei componente din vectorul $\phi(\mathbf{x})$.

Exercițiul 2: Evaluarea Overfitting-ului

Aveți un set de date de antrenare cu $n = 100$ exemple. V-ați antrenat trei modele de regresie polinomială:

Model	Grad Polinomial (M)	RMSE pe Date de Antrenare	RMSE pe Date de Testare
A	1	4.5	4.4
B	10	1.1	1.3
C	99	0.05	15.0

1. Care model suferă cel mai mult de **underfitting** (Bias ridicat)? Explicați de ce.
2. Care model suferă cel mai mult de **overfitting** (Varianță ridicată)? Explicați de ce.
3. Care model oferă cea mai bună **generalizare**?

Rezolvare 2:

1. **Underfitting (Model A):** Modelul A are cel mai mare RMSE pe ambele seturi (4.5 și 4.4). Acest lucru indică faptul că modelul (o simplă linie dreaptă) nu a fost suficient de complex pentru a se potrivi măcar datelor de antrenare, sugerând un **Bias ridicat**.
2. **Overfitting (Model C):** Modelul C ($M = 99$) se potrivește aproape perfect datelor de antrenare (RMSE=0.05), trecând practic prin fiecare punct. Cu toate acestea, performanța pe setul de testare este dezastruoasă (RMSE=15.0). Această diferență uriașă indică faptul că modelul a învățat zgomotul din datele de antrenare și are o **Varianță ridicată**, fiind cel mai supra-antrenat.
3. **Cea mai bună Generalizare (Model B):** Modelul B are o eroare de testare scăzută (1.3) și o diferență mică între eroarea de antrenare și cea de testare. Acest model reprezintă cel mai bun echilibru în **bias/variance trade-off** pentru datele furnizate.

1.5

Bine ați revenit! Ne afundăm acum într-un subiect vital pentru practicianul Machine Learning: **regularizarea**. Această lecție ne învață cum să controlăm complexitatea modelului pentru a lupta împotriva supra-antrenării (overfitting-ului), asigurând că modelele noastre generalizează bine pe date nevăzute.

Ghid de Studiu Complet: 1.5 Modele Liniare Regularizate (Regularized linear models)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Această lecție tratează problema **overfitting-ului**, care apare frecvent atunci când estimarea Maximum Likelihood (MLE) este aplicată pe modele liniare complexe (cum ar fi regresia polinomială de grad mare). Overfitting-ul se manifestă adesea prin **valori relativ mari** ale parametrilor (ponderilor). Soluția este introducerea **regularizării**, care constă în adăugarea unei penalizări la funcția de pierdere (MSE), forțând modelul să își mențină ponderile mici. Această tehnică este formalizată prin **estimarea Maximum a Posteriori (MAP)**, care găsește un **compromis** între a se potrivi bine datelor (likelihood) și a menține ponderi plauzibile (prior). Lecția detaliază cele trei forme principale de regularizare (Ridge, Lasso, Elastic Net) și introduce o metodă alternativă bazată pe timp: *Early Stopping*.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Cauza Overfitting:** MLE este predispus la overfitting, care se manifestă prin parametri cu **magnitudini relativ mari**.
2. **Principiul MAP:** Regularizarea este echivalentă cu **estimarea Maximum a Posteriori (MAP)**, care combină potrivirea datelor (log-likelihood) cu o penalizare bazată pe o distribuție prior pe ponderi (log-prior).
3. **Ridge Regression (ℓ_2):** Folosește **norma** ℓ_2 a ponderilor ca termen de regularizare. Forțează ponderile să fie **mici**, dar nu le setează la zero.
4. **Lasso Regression (ℓ_1):** Folosește **norma** ℓ_1. Trăsătura cheie este că **setează la zero** ponderile caracteristicilor mai puțin importante, realizând **selecția automată a caracteristicilor** (sparse model).
5. **Early Stopping:** O metodă de regularizare iterativă care oprește antrenarea (de exemplu, cu Gradient Descent) **imediat ce eroarea de validare atinge un minim** și începe să crească.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

1.5.1 De la Maximum Likelihood la Maximum a Posteriori (MAP)

- **Problema:** MLE este predispusă la overfitting.

- **Observație:** Când are loc overfitting-ul, magnitudinea valorilor parametrilor devine **relativ mare**.
- **Soluție (Distribuția Prior):** Pentru a atenua efectul ponderilor mari, se plasează o **distribuție prior** $p(\mathbf{w})$ pe parametri. Aceasta codifică explicit ce valori ale parametrilor sunt plauzibile *înainte* de observarea datelor.
  - Exemplu: Un prior Gaussian $\mathbf{w} \sim \mathcal{N}(0, 1)$ pe un singur parametru implică faptul că valorile parametrilor sunt așteptate să se afle în intervalul $[-2, 2]$ (două abateri standard în jurul mediei).
- **Estimarea MAP:** Odată ce un set de date este disponibil, în loc să se maximizeze likelihood-ul, se maximizează distribuția **posterioră** $p(\mathbf{w}|\mathbf{X}, \mathbf{y})$.
- **Formula Bayes:** Posteriorul se obține prin teorema lui Bayes:

$$p(\mathbf{w}|\mathbf{X}, \mathbf{y}) = \frac{p(\mathbf{y}|\mathbf{X}, \mathbf{w})p(\mathbf{w})}{p(\mathbf{y}|\mathbf{X})}$$

- **Compromisul:** Prin minimizarea log-posteriorului negativ:

$$-\operatorname{g} p(\mathbf{w}|\mathbf{X}, \mathbf{y}) = -\operatorname{g} p(\mathbf{y}|\mathbf{X}, \mathbf{w}) - \operatorname{g} p(\mathbf{w}) + \operatorname{g} p(\mathbf{y}|\mathbf{X}) \quad (10)$$

  Termenul $-\operatorname{g} p(\mathbf{y}|\mathbf{X}, \mathbf{w})$ este log-likelihood-ul negativ (MSE), iar $-\operatorname{g} p(\mathbf{w})$ este log-priorul negativ (termenul de regularizare). Estimarea MAP este un **compromis** între prior (plauzibilitate) și likelihood (potrivirea datelor).

1.5.2 Regresia Ridge (ℓ_2 Regularization)

- **Ipoteza Prior:** Se presupune că ponderile sunt distribuite **normal** (Gaussian) cu varianța , $\mathbf{w} \sim \mathcal{N}(0, {}^2)$.
- **Funcția de Pierdere (Loss Function)** corespunzătoare:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2n}(\mathbf{X}\mathbf{w} - \mathbf{y})^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) + \frac{1}{2}\lambda||\mathbf{w}||_2^2 \quad (7)$$

  - Această metodă se numește **ridge regression**.
  - Al doilea termen este termenul de regularizare, adăugat la funcția de pierdere.
  - $||\mathbf{w}||_2 = (\sum_{j=0}^{d} |w_j|^2)^{1/2}$ este **norma** ℓ_2 a lui $\mathbf{w}$.
- **Efect:** Regularizarea ℓ_2 forțează algoritmul de învățare nu numai să potrivească datele, ci și să mențină ponderile modelului cât mai **mici** posibil.
- **Ecuația Soluției Analitice (Closed-form):**

$$\mathbf{w}^* = (\mathbf{X}^\top\mathbf{X} + \lambda)^{-1}\mathbf{X}^\top\mathbf{y}$$

  - Unde $\lambda = \frac{\sigma^2}{{}^2}$.
- **Coborârea Gradientului (GD):** Pentru Ridge Regression, se adaugă pur și simplu $\lambda\mathbf{w}$ la vectorul gradientului neliniar:

$$\nabla_\mathbf{w}\mathcal{L}(\mathbf{w}) = \frac{1}{n}\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda\mathbf{w}$$

- **Hiperparametrul λ:** Controlează cât de mult dorim să regularizăm modelul.
  - Dacă $\lambda = 0$, Ridge Regression devine regresie liniară simplă.
  - Dacă λ este foarte mare, toate ponderile ajung foarte aproape de zero, iar rezultatul este o linie plată care trece prin media datelor.
- **Scalare:** Este important să se **scaleze datele** înainte de Ridge Regression, deoarece este sensibilă la scara caracteristicilor de intrare.

1.5.3 Regresia Lasso (ℓ_1 Regularization)

- **Ipoteza Prior:** Se presupune o **distribuție Laplace** pentru ponderi, $\mathbf{w} \sim \mathrm{ap}(0, b)$.
- **Funcția de Pierdere:** Adaugă un termen de regularizare bazat pe **norma** ℓ_1:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2n}(\mathbf{X}\mathbf{w} - \mathbf{y})^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) + \frac{1}{2}\lambda||\mathbf{w}||_1 \quad (11, 12)$$

  - Această metodă se numește **lasso regression** (Least Absolute Shrinkage and Selection Operator Regression).
  - $||\mathbf{w}||_1 = \sum_{j=0}^{d} |w_j|$ este **norma** ℓ_1 a lui $\mathbf{w}$.
- **Caracteristică Cheie (Feature Selection):** Lasso tinde să **elimine ponderile** caracteristicilor mai puțin importante, setându-le la zero.
  - Aceasta realizează automat **selecția caracteristicilor (*feature selection*)** și produce un **model rar (*sparse model*)**.
  - Modelele rare sunt mai ușor de înțeles de către om și sunt mai puțin predispuse la overfitting.
- **Optimizare cu Subgradient:** Funcția de pierdere Lasso nu este diferențiabilă când $w_j = 0$. Cu toate acestea, Coborârea Gradientului funcționează utilizând un **vector subgradient** $(\mathbf{w})$.

1.5.4 Elastic Net

- **Definiție:** Este o cale de mijloc între Ridge și Lasso Regression.
- **Termen de Regularizare:** Este un amestec simplu al termenilor de regularizare ℓ_1 și ℓ_2.
- **Funcția de Pierdere:**

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2n}(\mathbf{X}\mathbf{w} - \mathbf{y})^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) + r\lambda||\mathbf{w}||_1 + \frac{1-r}{2}\lambda||\mathbf{w}||_2^2$$

- **Controlul Amestecului:** Hiperparametrul $r \in$ controlează raportul de amestec.
  - $r = 0$ echivalează cu Ridge Regression.
  - $r = 1$ echivalează cu Lasso Regression.
- **Utilizare:** Elastic Net este preferat în general față de Lasso atunci când numărul de caracteristici este mai mare decât numărul de instanțe de antrenare sau când mai multe caracteristici sunt puternic corelate.

1.5.5 Early Stopping

- **Definiție:** O modalitate diferită de a regulariza algoritmii de învățare iterativi (cum ar fi Coborârea Gradientului).
- **Mecanism:** Antrenarea este oprită imediat ce **eroarea de validare atinge un minim**.
- **Semnal de Overfitting:** Când modelul învață, eroarea de antrenare continuă să scadă, dar la un moment dat (număr de epoci) eroarea de validare **începe să crească**; acest lucru indică faptul că modelul a început să se supra-antreneze.
- **Practică (pentru SGD/MBGD):** Deoarece curbele de eroare pot fi neregulate, se recomandă oprirea antrenării după ce eroarea de validare a fost deasupra minimului pentru o anumită perioadă, și apoi se revin la parametrii modelului de la punctul unde eroarea de validare era minimă.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Concept Dificil: Compromisul MAP și Priori (Log-Prior)

- **Explicație Simplificată (Ancorele Modelului):** Regresia liniară fără regularizare (MLE) este ca o barcă fără ancore: se mișcă liber pentru a se potrivi perfect fiecărui val (dată de antrenare), chiar și celor mici, de zgomot.
- **Log-Priorul Negativ (Penalizarea):** Când adăugăm regularizarea (Log-Priorul Negativ), este ca și cum am arunca o **ancoră** care trage barca înapoi spre o poziție centrală (ponderi mici, apropiate de zero).
- **MAP (Compromis):** Soluția optimă (MAP) nu este nici potrivirea perfectă a datelor (care ar duce la overfitting), nici menținerea ponderilor la zero, ci un **compromis**. Ancorele (priorii) limitează mișcarea barcii, forțând-o să găsească o poziție care este *suficient de aproape* de date, dar care menține modelul simplu (greutăți mici).

Diferența Esențială L1 vs. L2 (Geometrie)

- **L2 (Ridge):** Penalizează suma *pătratelor* ponderilor. Din punct de vedere geometric, constrângerea ℓ_2 este un **cerc**. Soluția optimă poate apărea oriunde pe cerc. Deoarece cercul nu are colțuri pe axe, ℓ_2 nu are o tendință generală de a produce ponderi zero.
- **L1 (Lasso):** Penalizează suma *valorilor absolute* ale ponderilor. Constrângerea ℓ_1 este un **diamant** (sau o cutie cu colțuri ascuțite). Minimele sunt adesea găsite la colțurile diamantului. Colțurile sunt punctele unde o pondere este zero (se află pe o axă), ceea ce explică de ce Lasso produce un model rar (sparse).

De ce trebuie scalate datele pentru Ridge/Lasso?

- *Asta se întâmplă deoarece:* Regularizarea penalizează magnitudinea absolută a ponderilor. Dacă o caracteristică are valori foarte mari (ex: 1.000.000) și alta valori foarte mici (ex: 0.1), algoritmul va acorda o pondere w_i foarte mică caracteristicii mari, doar

pentru a compensa scara, fără ca acea pondere mică să reflecte neapărat lipsa de importanță. Prin scalarea tuturor caracteristicilor la o scară similară, regularizarea devine **echitabilă** și penalizarea $||\mathbf{w}||$ reflectă cu adevărat complexitatea modelului.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Conexiunea cu Overfitting (Lecția 1.4):** Regularizarea este soluția directă la problema overfitting-ului indus de complexitatea modelului (cum ar fi gradele mari M în Regresia Polinomială). Overfitting-ul este sinonim cu Varianța Ridicată. Regularizarea adaugă un mic **Bias** (prin tragerea ponderilor spre zero), dar reduce **Varianța** modelului, îmbunătățind generalizarea.
- **Legătura între Gradient Descent și Regularizare:** Ambele metode (Ridge și Lasso) pot fi antrenate folosind Ecuația Normală modificată (doar Ridge) sau **Coborârea Gradientului**. Acest lucru este crucial, deoarece GD este singura metodă scalabilă pentru seturi de date și modele extrem de mari, făcând ca regularizarea să fie aplicabilă universal.
- **Alegerea Metodei (Ridge vs. Lasso vs. Elastic Net):**
  - **Ridge** este o alegere bună implicită (*good default*).
  - **Lasso** este de preferat dacă se suspectează că **doar câteva caracteristici sunt cu adevărat utile**, deoarece elimină caracteristicile inutile prin setarea ponderilor la zero (sparse model).
  - **Elastic Net** este recomandat în general față de Lasso, mai ales când $d \gg n$ (mai multe caracteristici decât exemple) sau când caracteristicile sunt puternic corelate, deoarece Lasso poate avea un comportament neregulat în aceste cazuri.

---

5. EXERCIȚII ȘI REZOLVARE

Exercițiul 1: Identificarea Obiectivului de Optimizare

Completați spațiile libere în cele două obiective de optimizare de mai jos și identificați tipul de regularizare asociat fiecărui caz.

1. **Obiectivul I:** Minimizarea $(\text{MSE ermenu}) + \frac{1}{2}\lambda \sum_j w_j^2$.
2. **Obiectivul II:** Minimizarea $(\text{MSE ermenu}) + \frac{1}{2}\lambda \sum_j |w_j|$.

Rezolvare 1:

1. **Obiectivul I** folosește norma ℓ_2 (suma pătratelor). Acesta corespunde **Ridge Regression**.
2. **Obiectivul II** folosește norma ℓ_1 (suma valorilor absolute). Acesta corespunde **Lasso Regression**.

Exercițiul 2: Efectul Hiperparametrului λ

Ce se întâmplă cu modelul de Regresie Ridge în următoarele cazuri extreme, în funcție de hiperparametrul λ?

1. λ este setat la 0 ($\lambda = 0$).
2. λ este setat la o valoare foarte mare ($\lambda \to \infty$).

Rezolvare 2:

1. Dacă $\lambda = 0$, termenul de regularizare devine zero. **Ridge Regression se reduce la regresia liniară simplă** (ne-regularizată).
2. Dacă λ este foarte mare, penalizarea domină. Modelul va încerca să minimizeze costul de a avea ponderi mari. Toate ponderile ajung **foarte aproape de zero**, iar rezultatul este o linie plată care aproximează media datelor.

Exercițiul 3: Avantajele Selecției de Caracteristici

De ce este considerată **Lasso Regression** superioară (sau cel puțin preferabilă) Regresiei Ridge în scenariile în care se suspectează că majoritatea caracteristicilor de intrare sunt irelevante?

Rezolvare 3:

Lasso Regression este preferabilă în acest caz, deoarece utilizează regularizarea ℓ_1 care are proprietatea de a **seta automat la zero** ponderile caracteristicilor considerate irelevante. Acest proces este numit **selecția automată a caracteristicilor (*feature selection*)**. Prin eliminarea caracteristicilor inutile, Lasso produce un **model rar (*sparse model*)** care este mai simplu de interpretat de către oameni și mai puțin probabil să se supra-antreneze. Ridge Regression, pe de altă parte, doar micșorează ponderile, fără a le anula.

1.6

Absolut! Trecem acum la **Regresia Logistică**, un algoritm fundamental care, în ciuda numelui său, este folosit pentru **clasificare** și nu pentru regresie. Este puntea noastră de legătură de la modelele liniare simple la problemele de decizie.

Ghid de Studiu Complet: 1.6 Regresia Logistică (Logistic Regression)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Această lecție introduce **Regresia Logistică** (cunoscută și sub numele de **Logit Regression**), un algoritm de clasificare binară care estimează **probabilitatea** ca o instanță să aparțină unei anumite clase. Spre deosebire de Regresia Liniară care produce o valoare

continuă, Regresia Logistică aplică **funcția logistică** (sau **sigmoidă**) sumei ponderate a caracteristicilor, forțând rezultatul să se încadreze între 0 și 1. Această probabilitate estimată ($\hat{p}$) permite clasificarea binară (clasa pozitivă "1" sau clasa negativă "0"). Importanța acestei lecții este că demonstrează trecerea de la probleme de regresie la probleme de clasificare, introducând o **funcție de pierdere specifică (Log Loss)** care este optimizabilă și garantează găsirea minimului global.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Scopul (Clasificare Binară):** Regresia Logistică este folosită pentru a estima **probabilitatea** ca o instanță să aparțină clasei pozitive (etichetată "1").
2. **Mecanismul de Transformare:** Utilizează o **sumă ponderată liniar** a caracteristicilor, dar aplică rezultatul funcției **Sigmoid** (sau Logistic) $\sigma(t)$, care mapează orice valoare reală t la un număr între 0 și 1.
3. **Decizia de Clasificare:** Dacă probabilitatea estimată $\hat{p}$ este **mai mare de 50% ($\hat{p} \geq 0.5$)**, instanța este clasificată ca aparținând **clasei pozitive ("1")**; altfel, clasei negative ("0").
4. **Funcția de Pierdere (Log Loss):** Modelul este antrenat prin minimizarea unei funcții de pierdere numită **Log Loss** (sau Binary Cross-Entropy), care penalizează puternic probabilitățile incorecte.
5. **Optimizarea:** Deși **nu există o ecuație analitică** (similară Ecuației Normale) pentru găsirea soluției, funcția de pierdere este **convexă**, garantând că algoritmi de optimizare precum Coborârea Gradientului vor găsi minimul global.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

1.6 Regresia Logistică

- **Utilizare:** Regresia Logistică este un algoritm comun de regresie utilizat pentru **clasificare** (și vice versa, unele algoritmi de regresie pot fi folosiți pentru clasificare).
- **Estimarea Probabilității:** Este utilizată pentru a estima probabilitatea ($\hat{p}$) ca o instanță să aparțină unei clase specifice (ex: probabilitatea ca un email să fie spam).
- **Clasificare Binară:** Rezultatul este o clasificare binară:
  - Clasa Pozitivă: Etichetată "1", prezisă dacă $\hat{p} \geq 0.5$.
  - Clasa Negativă: Etichetată "0", prezisă altfel ($\hat{p} < 0.5$).

Modelul și Funcția Sigmoid (Logistică)

- Similar cu regresia liniară, modelul logistic calculează o **sumă ponderată** a caracteristicilor de intrare (plus un termen de bias).
- Rezultatul acestei sume ponderate este denumit adesea **scorul** t.
- În loc să scoată rezultatul direct ca în regresia liniară, modelul aplică scorului t **funcția logistică** sau **Sigmoid**, notată $\sigma(\cdot)$:

$$\hat{p} = \sigma(\mathbf{w}^\top \mathbf{x}) \quad \text{unde } t = \mathbf{w}^\top \mathbf{x}$$

- **Definiția Funcției Sigmoid (Ecuația de bază):**

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$

  - Funcția Sigmoid este în formă de S și produce întotdeauna un număr între 0 și 1.
  - Funcția Sigmoid produce $\sigma(t) < 0.5$ când $t < 0$, și $\sigma(t) \geq 0.5$ când $t \geq 0$. Astfel, prezice 1 dacă $\mathbf{w}^\top \mathbf{x}$ este pozitiv și 0 dacă este negativ.

Logit (Log-Odds)

- Scorul t este adesea numit **logit**.
- Logit-ul este definit ca **inversa funcției Sigmoid**: $\mathrm{git}(\hat{p}) = \mathrm{g}(\hat{p}/(1-\hat{p}))$.
- Logit-ul este, de asemenea, numit **log-odds**, deoarece este logaritmul raportului dintre probabilitatea estimată pentru clasa pozitivă și probabilitatea estimată pentru clasa negativă.
- Calculând logit-ul probabilității estimate $\hat{p}$, rezultatul este scorul t.

Funcția de Pierdere (Log Loss / Cross-Entropy)

- **Obiectivul Antrenării:** Setarea vectorului de ponderi $\mathbf{w}$ astfel încât modelul să estimeze probabilități **mari** pentru instanțele pozitive ($y = 1$) și probabilități **mici** pentru instanțele negative ($y = 0$).
- **Funcția de Pierdere pentru un Singur Exemplu x (Ecuație pe cazuri):**

$$l(\mathbf{w}) = \begin{cases} -\mathrm{g}\,\hat{p}, & \text{dacă } y = 1 \\ -\mathrm{g}(1-\hat{p}), & \text{dacă } y = 0 \end{cases}$$

- **Logica Pierderii:**
  - Dacă $y = 1$ (instanță pozitivă) și $\hat{p} \to 0$, $-\mathrm{g}\,\hat{p}$ crește foarte mare (pierdere mare).
  - Dacă $y = 0$ (instanță negativă) și $\hat{p} \to 1$, $-\mathrm{g}(1-\hat{p})$ crește foarte mare (pierdere mare).
  - Pierderea tinde spre 0 când predicția este corectă (ex: $y = 1$ și $\hat{p} \to 1$).
- **Funcția de Pierdere în Expresie Unică (Ecuația 12):** Poate fi scrisă într-o singură expresie compactă:

$$l(\mathbf{w}) = -y\,\mathrm{g}\,\hat{p} - (1-y)\,\mathrm{g}(1-\hat{p})$$

- **Funcția de Cost Globală ($\mathcal{L}(\mathbf{w})$):** Media pierderilor pe întregul set de antrenare de n instanțe:

$$\mathcal{L}(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^{n} \left[y^{(i)}\,\mathrm{g}\,\hat{p}^{(i)} + (1-y^{(i)})\,\mathrm{g}(1-\hat{p}^{(i)}) \right] \quad (12)$$

Optimizarea și Regularizarea

- **Soluție Analitică (Lipsă): Nu există o ecuație analitică** (nu există un echivalent al Ecuației Normale) pentru a calcula valoarea $\mathbf{w}$ care minimizează Log Loss.
- **Optimizare Iterativă:** Vestea bună este că funcția de pierdere este **convexă**.
- **Găsirea Minimului Global:** Coborârea Gradientului (sau alt algoritm de optimizare) este **garantată** să găsească minimul global (presupunând că rata de învățare nu este prea mare și antrenarea este suficient de lungă).
- **Gradientul Funcției de Pierdere:** Gradientul are o formă similară cu Ecuația (8) pentru Regresia Liniară:

$$\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}) = \frac{1}{n}\mathbf{X}^{\top}(\sigma(\mathbf{X}\mathbf{w}) - \mathbf{y})$$

- **Regularizare:** Similar altor modele liniare, Regresia Logistică poate fi regularizată folosind penalizări ℓ_1 sau ℓ_2.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Concept Dificil: Funcția Sigmoid (Logistică)

- **Analogie Simplificată (Termostatul cu Prag):** Regresia Liniară funcționează ca un termometru - dă o citire continuă (ex: -5, 0, 50, 100). Regresia Logistică ia acea citire și o trece printr-un sistem Sigmoid, ca un termostat (pragul de 0).
  - Dacă temperatura (scorul t) este **mult sub 0**, probabilitatea este aproape 0.
  - Dacă temperatura este **mult peste 0**, probabilitatea este aproape 1.
  - Numai în jurul valorii $t = 0$ se face tranziția (la $\hat{p} = 0.5$), transformând o valoare continuă într-o probabilitate discretă.

Termen Tehnic: Logit/Log-Odds

- **Context:** Dacă $\hat{p}$ este probabilitatea ca un eveniment să aibă loc, atunci *odds* (șansele) este raportul $\hat{p}/(1-\hat{p})$.
- **Exemplu:** Dacă $\hat{p} = 0.8$ (80% șanse), șansele (odds) sunt $0.8/0.2 = 4$. Logit-ul este $\mathrm{g}(4)$.
- *Asta se întâmplă deoarece:* Logit-ul este simetric în jurul lui zero. Dacă $\hat{p} = 0.5$ (50% șanse), $odd = 1$ și logit-ul este $\mathrm{g}(1) = 0$. Dacă $\hat{p} = 0.9$, logit-ul e pozitiv; dacă $\hat{p} = 0.1$, logit-ul e negativ. Acest lucru aliniază perfect logit-ul cu scorul liniar $t = \mathbf{w}^{\top}\mathbf{x}$.

Funcția de Pierdere (Log Loss)

- *Asta se întâmplă deoarece:* Nu putem folosi Eroarea Pătratică Medie (MSE) pentru clasificare. De ce? Deoarece funcția Sigmoid introduce neliniaritatea, iar suprafața de pierdere MSE nu ar mai fi convexă. Dacă nu ar fi convexă, algoritmi ca GD ar fi blocați în minime locale, neputând garanta găsirea celei mai bune soluții.

- **Log Loss (Cross-Entropy)** asigură că suprafața de pierdere **rămâne convexă**, făcând posibilă optimizarea.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Conexiunea cu Regresia Liniară:** Deși are un nume similar, Regresia Logistică *include* Regresia Liniară în nucleul său. Calculul scorului $t = \mathbf{w}^\top \mathbf{x}$ este exact același proces ca predicția într-un model liniar. Diferența esențială este **funcția de activare** (Sigmoid) aplicată rezultatului, transformând ieșirea continuă într-o probabilitate.
- **Conexiunea cu Optimizarea (GD):** Deoarece nu are o soluție analitică (fără Ecuația Normală), Regresia Logistică **depinde în totalitate** de metode iterative de optimizare precum Batch, Stochastic sau Mini-Batch Gradient Descent (discutate anterior).
- **Generalizarea Clasificării:** Log Loss (sau Binary Cross-Entropy) nu este doar pierderea pentru Regresia Logistică, ci este **funcția de pierdere standard** folosită pentru antrenarea oricărui model de clasificare binară, inclusiv rețelele neuronale adânci (pe care le vom studia mai târziu).

---

5. EXERCIȚII ȘI REZOLVARE

Vom folosi modelul de rezolvare matricială prezentat în sursa.

Exercițiul 1: Interpretarea Sigmoid și Decizia de Clasificare

Presupunem că un model de Regresie Logistică a fost antrenat și, pentru o nouă instanță de date, calculează scorul liniar $t = \mathbf{w}^\top \mathbf{x}$.

1. Dacă scorul t este -2, calculați probabilitatea estimată $\hat{p}$ (folosind doar intervalul de valori).
2. Dacă $\hat{p} = 0.75$, care este predicția finală $\hat{y}$ (clasa)?
3. Dacă $\hat{p} = 0.30$, care este predicția finală $\hat{y}$?

Rezolvare 1:

1. Funcția Sigmoid $\sigma(t)$ produce $\sigma(t) < 0.5$ când $t < 0$. Deoarece $t = -2$ (negativ), probabilitatea estimată $\hat{p}$ va fi **mai mică de 0.5** (sau mai mică de 50%).
2. Deoarece $\hat{p} = 0.75$ este ≥ 0.5 (mai mare de 50%), modelul prezice că instanța aparține **clasei pozitive (etichetată "1")**.
3. Deoarece $\hat{p} = 0.30$ este < 0.5 (mai mică de 50%), modelul prezice că instanța aparține **clasei negative (etichetată "0")**.

Exercițiul 2: Evaluarea Pierderii (Log Loss)

Calculați Log Loss $l(\mathbf{w})$ pentru un singur exemplu i în următoarele două scenarii extreme, bazându-vă pe formula pe cazuri:

$$\mathrm{g\ ss}\, l(\mathbf{w}) = \quad -\mathrm{g}\,\hat{p}, \quad \text{dacă } y = 1 \ -\mathrm{g}(1-\hat{p}), \quad \text{dacă } y = 0$$

1. Scenariul A: Instanța este pozitivă ($y = 1$), dar modelul prezice o probabilitate foarte mică, $\hat{p} = 0.001$.
2. Scenariul B: Instanța este negativă ($y = 0$), iar modelul prezice o probabilitate foarte mică, $\hat{p} = 0.001$.

Rezolvare 2:

- *Notă: Deși sursa nu oferă o bază de logaritm, vom folosi logaritmul natural (ln) pentru interpretare, deoarece în ML logaritmul este tipic sau 2.*

1. **Scenariul A ($y = 1, \hat{p} = 0.001$):**

   $$l(\mathbf{w}) = -\mathrm{g}(0.001)$$

   Deoarece logaritmul unui număr foarte mic se apropie de $-\infty$, **Log Loss devine o valoare pozitivă foarte mare**. Modelul este penalizat sever, ceea ce confirmă că funcția de pierdere forțează modelul să estimeze probabilități mari pentru instanțele pozitive.
2. **Scenariul B ($y = 0, \hat{p} = 0.001$):**

   $$l(\mathbf{w}) = -\mathrm{g}(1-\hat{p}) = -\mathrm{g}(1-0.001) = -\mathrm{g}(0.999)$$

   Deoarece logaritmul unui număr apropiat de 1 se apropie de 0, **Log Loss este foarte mic** (aproape de zero). Acest lucru indică faptul că modelul a prezis corect că instanța aparține clasei negative (probabilitate mică).

Exercițiul 3: Necesarul de Optimizare Iterativă

De ce nu putem folosi soluția analitică (Ecuația Normală) pentru a calcula direct ponderile optime $\mathbf{w}^*$ în Regresia Logistică, așa cum facem în Regresia Liniară?

Rezolvare 3:

Conform surselor, **nu există o ecuație analitică cunoscută (closed-form equation)** echivalentă Ecuației Normale care să minimizeze funcția de pierdere Log Loss. Acest lucru se datorează naturii neliniare a funcției Sigmoid. Prin urmare, Regresia Logistică trebuie antrenată folosind **metode iterative de optimizare** (cum ar fi Coborârea Gradientului), profitând de faptul că funcția de pierdere este **convexă** și permite găsirea minimului global.

1.7

Bună ziua! Continuăm explorarea modelelor liniare trecând de la clasificarea binară la sarcina mai complexă a **clasificării multi-clasă**. Această lecție introduce **Regresia Softmax**, care este o generalizare directă și puternică a Regresiei Logistice.

Ghid de Studiu Complet: 1.7 Regresia Softmax (Softmax Regression)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Această lecție se concentrează pe **Regresia Softmax** (cunoscută și sub numele de **Regresie Logistică Multinomială**), o metodă care generalizează Regresia Logistică pentru a suporta **multiple clase (multiclass)** direct, eliminând necesitatea antrenării și combinării mai multor clasificatori binari. Ideea centrală este că modelul calculează un scor liniar pentru **fiecare clasă** în parte și apoi transformă aceste scoruri în probabilități prin aplicarea funcției **Softmax**. Regresia Softmax folosește **Cross-Entropy (Entropia Încrucișată)** ca funcție de pierdere, care este funcția standard pentru măsurarea neconcordanței (*mismatch*) dintre distribuțiile de probabilitate estimate și cele țintă. Este esențial de reținut că Softmax este utilizată doar pentru **clase mutual exclusive** (mutually exclusive classes).

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Generalizare Multi-Clasă:** Regresia Softmax generalizează Regresia Logistică pentru probleme de clasificare cu q clase.
2. **Mecanismul de Scorul:** Pentru o instanță $\mathbf{x}$, modelul calculează un **scor liniar** $o_k(\mathbf{x}) = \mathbf{w}_k^\top \mathbf{x}$ pentru fiecare clasă $k = 1, \ldots, q$. Fiecare clasă are propriul său vector de parametri dedicat $\mathbf{w}_k$.
3. **Funcția Softmax:** Această funcție (numită și *normalized exponential*) transformă scorurile $o_k(\mathbf{x})$ în probabilități estimate $\hat{p}_k$.
4. **Decizia și Aplicabilitatea:** Modelul prezice clasa k cu **cea mai mare probabilitate estimată** (adică scorul cel mai mare) și este utilizat **doar** pentru **clase mutual exclusive** (mutually exclusive classes).
5. **Funcția de Pierdere:** Antrenarea se realizează prin minimizarea funcției **Cross-Entropy** (entropia încrucișată), care penalizează modelul atunci când estimează o probabilitate mică pentru clasa țintă.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

1.7 Regresia Softmax

- **Definiție:** Generalizarea modelului de regresie logistică pentru a suporta direct clase multiple. Se mai numește **regresie logistică multinomială (*multinomial logistic regression*)**.
- **Structura Problemei:** Se presupune că avem o problemă de clasificare cu q clase.

Modelul de Scorul și Parametrii

- **Scorul Liniar:** Pentru o instanță $\mathbf{x}$, modelul calculează un scor $o_k(\mathbf{x})$ pentru fiecare clasă k.

$$o_k(\mathbf{x}) = \mathbf{w}_k^\top \mathbf{x}$$

  - Această ecuație este identică cu ecuația de predicție a regresiei liniare.
- **Ponderi (Weights):** Fiecare clasă k are propriul său **vector de parametri dedicat** $\mathbf{w}_k \in \mathbb{R}^d$.
- **Matricea de Parametri:** Toți vectorii de parametri sunt stocați sub formă de coloane într-o **matrice de parametri** $\in \mathbb{R}^{d \times q}$:

$$= [\mathbf{w}_1 \quad \mathbf{w}_2 \quad \dots \quad \mathbf{w}_q]$$

- Vectorul de scoruri $(\mathbf{x})$ conține scorurile pentru fiecare clasă pentru instanța $\mathbf{x}$.

Funcția Softmax (Normalized Exponential)

- Scorurile $(\mathbf{x})$ sunt transformate în probabilități estimate $\hat{p}_k$ prin aplicarea **funcției Softmax**:

$$\hat{p}_k = \mathrm{sftmax}((\mathbf{x}))_k = \frac{\exp(o_k(\mathbf{x}))}{\sum_{j=1}^{q} \exp(o_j(\mathbf{x}))}$$

- Funcția Softmax calculează exponențiala fiecărui scor și apoi normalizează aceste valori prin împărțirea la suma tuturor exponențialelor.
- Scorurile $o(\mathbf{x})$ sunt denumite în general **logits** sau **log-odds** (deși sunt de fapt *unnormalized log-odds*).

Predicția Finală și Domeniul de Aplicare

- Clasificatorul Softmax prezice clasa cu **cea mai mare probabilitate estimată**:

$$\hat{y} = \arg\max_k \mathrm{sftmax}((\mathbf{x}))_k = \arg\max_k o_k(\mathbf{x}) = \arg\max_k \mathbf{w}_k^\top \mathbf{x}$$

  - Operatorul $\arg\max_k$ returnează valoarea lui k care maximizează probabilitatea (sau, echivalent, scorul $o_k(\mathbf{x})$).
- **Atenție:** Clasificatorul Softmax prezice **o singură clasă la un moment dat** (multiclass, nu multioutput).
- Softmax trebuie utilizat numai cu **clase mutual exclusive** (mutually exclusive classes), cum ar fi diferite tipuri de plante, nu pentru a detecta mai multe persoane într-o singură imagine.

Funcția de Pierdere: Cross-Entropy (Entropia Încrucișată)

- **Obiectivul Antrenării:** Modelul trebuie să estimeze o **probabilitate mare** pentru clasa țintă și, în consecință, o probabilitate mică pentru celelalte clase.

- **Minimizarea Pierderii:** Se minimizează funcția de pierdere **Cross-Entropy (entropia încrucișată)**.
- **Formula Funcției de Pierdere Globală $\mathcal{L}()$ (Ecuația):**

$$\mathcal{L}() = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{q} y_k^{(i)} \, \mathrm{g}\, \hat{p}_k^{(i)}$$

  - $y_k^{(i)}$ este probabilitatea țintă (de obicei 1 sau 0) ca instanța i să aparțină clasei k.
- **Echivalența cu Regresia Logistică:** Când există doar **două clase ($q = 2$)**, funcția de pierdere Cross-Entropy este echivalentă cu funcția de pierdere a Regresiei Logistice (Ecuația 12 din lecția anterioară).
- **Măsura Mismatch:** Cross-Entropy (p, q) măsoară **neconcordanța (*mismatch*)** dintre distribuțiile de probabilitate q (estimate) și p (țintă).
  - Cea mai mică Cross-Entropy posibilă este atinsă când $p = q$.
- **Gradientul Cross-Entropy:** Gradientul funcției de pierdere în raport cu vectorul $\mathbf{w}_k$ este dat de:

$$\nabla_{\mathbf{w}_k}\mathcal{L}() = \frac{1}{n}\mathbf{X}^\top(\mathscr{k} - \mathbf{y}_k) = \frac{1}{n}\mathbf{X}^\top(\mathrm{sftmax}((\mathbf{x}))_k - \mathbf{y}_k)$$

- **Optimizare:** Vectorul gradient pentru întreaga matrice de parametri poate fi calculat și folosit în **Coborârea Gradientului (Gradient Descent)** sau orice alt algoritm de optimizare pentru a minimiza funcția de pierdere.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Concept Dificil: Funcția Softmax vs. Sigmoid

- **Sigmoid (Binară):** Funcționează ca un filtru binar. Ia un singur scor și îl transformă într-o probabilitate între 0 și 1. Probabilitatea clasei 0 este pur și simplu $1 - p(\text{casa } 1)$.
- **Softmax (Multi-Clasă):** Ia un **vector de scoruri** (câte unul pentru fiecare clasă) și le transformă pe toate într-un **vector de probabilități**. Important este că aceste probabilități sunt **normalizate**, adică suma lor este întotdeauna 1.
- *Asta se întâmplă deoarece:* Exponențiala asigură că scorurile negative sunt transformate în numere pozitive, iar normalizarea (împărțirea la suma exponențialelor) le transformă în probabilități care sumează la 1, îndeplinind cerințele unei distribuții de probabilitate.

Termen Tehnic: Argmax

- **Explicație Simplificată (ELI5):** Imaginați-vă că aveți un coș cu mere de culori diferite și vreți să știți ce culoare predomină.
  - Softmax vă dă probabilitatea fiecărei culori (ex: 0.2 roșu, 0.7 verde, 0.1 galben).

- **Argmax** (Argument Maxim) nu returnează valoarea maximă (0.7), ci returnează **indicele** (sau clasa) care deține acea valoare maximă (în acest caz, "verde").
- Matematic, $\arg\max_k$ ne dă clasa k cu scorul (sau probabilitatea) cel mai mare.

Cross-Entropy ca Măsură de Pierdere

- **Context:** Cross-Entropy () penalizează modelul atunci când predicțiile sunt diferite de realitate.
- *Asta se întâmplă deoarece:* Deoarece logaritmul unui număr mic (probabilitate mică) este un număr negativ mare, atunci $-\mathrm{g}(\hat{p})$ este un număr pozitiv mare. Dacă modelul a estimat o probabilitate mică ($\hat{p}_k$) pentru clasa țintă ($y_k = 1$), log loss-ul (și deci Cross-Entropy) devine foarte mare, penalizând puternic modelul.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Generalizare Liniară:** Regresia Softmax este extinderea logică a celor două concepte anterioare:
  - Preia structura liniară de scoring de la **Regresia Liniară** ($o_k(\mathbf{x}) = \mathbf{w}_k^\top \mathbf{x}$).
  - Preia funcția de pierdere bazată pe logaritmi de la **Regresia Logistică** (Log Loss), pe care o generalizează sub forma **Cross-Entropy**.
- **Dependența de Optimizare Iterativă:** Similar Regresiei Logistice, Softmax Regression **nu are o soluție analitică** (fără Ecuația Normală). Prin urmare, antrenarea ei depinde în totalitate de algoritmi iterativi de optimizare, în special Coborârea Gradientului.
- **Conexiunea cu Rețelele Neuronale Adânci:** Stratul de ieșire (output layer) al oricărei rețele neuronale profunde care efectuează clasificarea multi-clasă folosește întotdeauna **funcția de activare Softmax** și **pierderea Cross-Entropy** pentru antrenare. Înțelegerea Softmax-ului este fundamentală pentru DL.

---

5. EXERCIȚII ȘI REZOLVARE

Vom folosi notațiile și formulele din surse, urmând modelul de rezolvare matricială (deși calculul numeric direct al ponderilor Softmax este complex, ne vom concentra pe înțelegerea mecanismului de scor și predicție).

Exercițiul 1: Calculul Probabilității Softmax

Presupunem că un model Softmax a calculat scorurile $o(\mathbf{x})$ pentru o instanță $\mathbf{x}$ în 3 clase ($q = 3$):

- Clasa 1 (Câine): $o_1 = 2$
- Clasa 2 (Pisică): $o_2 = 1$

- Clasa 3 (Pasăre): $o_3 = 0$

Calculați probabilitatea estimată $\hat{p}_1$ pentru Clasa 1, folosind formula Softmax:

$$\hat{p}_k = \frac{\exp(o_k(\mathbf{x}))}{\sum_{j=1}^{q} \exp(o_j(\mathbf{x}))}$$

Nota: Folosiți o aproximare pentru $\exp(1) \approx 2.72$ *și* $\exp(2) \approx 7.39$, $\exp(0) = 1$.

Rezolvare 1:

Pasul 1: Calculul Numitorului (Suma Exponențialelor)

$$\sum_{j=1}^{3} \exp(o_j) = \exp(o_1) + \exp(o_2) + \exp(o_3)$$

$$\sum = \exp(2) + \exp(1) + \exp(0) \approx 7.39 + 2.72 + 1.00 = 11.11$$

Pasul 2: Calculul Probabilității $\hat{p}_1$

$$\hat{p}_1 = \frac{\exp(o_1)}{\sum} = \frac{\exp(2)}{11.11} \approx \frac{7.39}{11.11} \approx 0.665$$

Pasul 3: Predicția Finală Scorul cel mai mare a fost $o_1 = 2$. Probabilitatea cea mai mare este $\hat{p}_1 \approx 0.665$. Modelul prezice clasa **Câine (Clasa 1)**.

Exercițiul 2: Interpretarea Logicii de Pierdere (Cross-Entropy)

Un model Softmax face două predicții pentru două exemple, ambele aparținând clasei țintă $k = 3$ ($y_3 = 1$, $y_1 = 0, y_2 = 0$):

1. **Predicția A:** $\hat{p}_3 = 0.95$ (foarte corect).
2. **Predicția B:** $\hat{p}_3 = 0.10$ (foarte greșit).

Folosind porțiunea de formulă relevantă din Cross-Entropy pentru un singur exemplu i și o singură clasă k:

$$l^{(i)} = -\sum_{k} y_k^{(i)} \, \mathrm{g} \, \hat{p}_k^{(i)}$$

Concentrându-ne doar pe termenul pozitiv ($y_3 = 1$): $-\mathrm{g}\,\hat{p}_3$. Care predicție va genera o pierdere (loss) mai mare și de ce?

Rezolvare 2:

1. **Predicția A (Corectă):** Pierderea $\approx -\mathrm{g}(0.95)$. Deoarece 0.95 este foarte aproape de 1, $\mathrm{g}(0.95)$ este aproape de 0. Pierderea este **foarte mică**.
2. **Predicția B (Gresită):** Pierderea $\approx -\mathrm{g}(0.10)$. Deoarece 0.10 este mult mai mic, $\mathrm{g}(0.10)$ este un număr negativ mai mare. Prin aplicarea negativului, pierderea este un **număr pozitiv mare**.

Concluzie: Predicția B (0.10) generează o pierdere mult mai mare. Aceasta demonstrează cum funcția Cross-Entropy **penalizează sever** modelul atunci când estimează o probabilitate mică pentru clasa țintă, forțându-l să se corecteze în iterațiile următoare.

Exercițiul 3: Aplicabilitatea Softmax

Pentru care dintre următoarele sarcini de clasificare ar fi adecvată utilizarea Regresiei Softmax? Justificați pe baza sursei.

A) Detectarea tuturor obiectelor dintr-o singură imagine (ex: o pisică și un câine sunt prezente simultan). B) Clasificarea unui email ca fiind "Serviciu", "Personal" sau "Junk".

Rezolvare 3:

Opțiunea corectă este **B) Clasificarea unui email ca fiind "Serviciu", "Personal" sau "Junk"**.

Justificare: Regresia Softmax trebuie utilizată numai cu **clase mutual exclusive** (mutually exclusive classes).

- În cazul B), un email nu poate fi simultan "Junk" și "Personal"; clasele se exclud reciproc, făcând Softmax adecvată.
- În cazul A), detectarea mai multor obiecte într-o singură imagine este o problemă **multioutput** (multi-etichetă), iar Softmax este un clasificator **multiclass** care prezice doar o singură clasă la un moment dat.

1.8

Acționând ca un profesor universitar, vă voi pregăti un set de exerciții detaliate, bazate pe structura și formulele prezente în materialele sursă.

Exerciții Asemănătoare Modelelor Liniare

A. Regresia Liniară (Linear Regression)

Exercițiul A.1: Calculul Ponderilor cu Ecuația Normală

Presupunem că avem un set de date format din trei puncte în planul 2D:

$$\text{Dataset } = (x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), (x^{(3)}, y^{(3)}) = (1,3), (2,5), (3,7)$$

Vrem să estimăm următorul model de regresie liniară:

$$\hat{y} = w_1 x + w_0$$

Calculați ponderile optime $\mathbf{w}^* = [w_0 \; w_1]$ folosind Ecuația Normală.

Rezolvare A.1:

Formulele necesare: Matricea de Design $\mathbf{X}$, Vectorul Țintă $\mathbf{y}$, și Ecuația Normală:

$$\mathbf{X} = \begin{bmatrix} 1 & x^{(1)} & 1 & x^{(2)} & 1 & x^{(3)} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y^{(1)} & y^{(2)} & y^{(3)} \end{bmatrix}, \quad \mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Pasul 1: Construirea Matricelor $\mathbf{X}$ și $\mathbf{y}$

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 2 & 1 & 3 \end{bmatrix} \in \mathbb{R}^{3\times 2}, \quad \mathbf{y} = \begin{bmatrix} 3 & 5 & 7 \end{bmatrix} \in \mathbb{R}^{3\times 1}$$

Pasul 2: Calculul $\mathbf{X}^\top \mathbf{X}$ și $\mathbf{X}^\top \mathbf{y}$

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 2 & 1 & 3 \end{bmatrix} = \begin{bmatrix} 3 & 6 & 6 & 14 \end{bmatrix}$$

$$\mathbf{X}^\top \mathbf{y} = \begin{bmatrix} 1 & 1 & 1 & 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 3 & 5 & 7 \end{bmatrix} = \begin{bmatrix} 3+5+7 & 3+(2\cdot 5)+(3\cdot 7) \end{bmatrix} = \begin{bmatrix} 15 & 34 \end{bmatrix}$$

Pasul 3: Calculul $\mathbf{w}^*$ Trebuie calculată inversa matricei 2×2: $(\mathbf{X}^\top \mathbf{X})^{-1}$.

$$\det(\mathbf{X}^\top \mathbf{X}) = (3\cdot 14) - (6\cdot 6) = 42 - 36 = 6$$

$$(\mathbf{X}^\top \mathbf{X})^{-1} = \frac{1}{6} \begin{bmatrix} 14 & -6 & -6 & 3 \end{bmatrix} = \begin{bmatrix} 14/6 & -1 & -1 & 3/6 \end{bmatrix} = \begin{bmatrix} 7/3 & -1 & -1 & 1/2 \end{bmatrix}$$

$$\mathbf{w}^* = \begin{bmatrix} 7/3 & -1 & -1 & 1/2 \end{bmatrix} \begin{bmatrix} 15 & 34 \end{bmatrix} = \begin{bmatrix} (7/3\cdot 15) - 34 & (-1\cdot 15) + (1/2\cdot 34) \end{bmatrix} = \begin{bmatrix} 35-34 & -15+17 \end{bmatrix} = |$$

Ponderile optime sunt $\mathbf{w}^* = \begin{bmatrix} w_0 & w_1 \end{bmatrix} = \begin{bmatrix} 1 & 2 \end{bmatrix}$. Modelul este $\hat{y} = 2x + 1$.

Exercițiul A.2: Interpretarea Ponderilor

Pentru modelul obținut la Exercițiul A.1 ($\hat{y} = 2x + 1$):

1. Care este semnificația numerică a ponderii w_0?
2. Care este semnificația numerică a ponderii w_1?

Rezolvare A.2:

1. $w_0 = 1$ reprezintă termenul de **bias** (interceptare). Aceasta este valoarea prezisă $\hat{y}$ atunci când **caracteristica** x **este zero**.
2. $w_1 = 2$ reprezintă **panta** dreptei. Indică faptul că, pentru fiecare creștere cu o unitate a caracteristicii x, valoarea prezisă $\hat{y}$ crește cu **două unități**.

B. Coborârea Gradientului (Gradient Descent)

Exercițiul B.1: Aplicarea Actualizării Batch Gradient Descent (BGD)

Folosind setul de date din sursă:

$$\text{Dataset } = (1,1), (2,-1), (3,2)$$

și modelul $\hat{y} = w_1 x + w_0$.

Asumând că ponderile inițiale sunt $\mathbf{w}^{(0)} = [0\ 0]$ și rata de învățare este $\eta = 0.1$. Scrieți formula de actualizare pentru $\mathbf{w}$ și aplicați-o numeric pentru a calcula $\mathbf{w}^{(1)}$.

Rezolvare B.1:

Formule necesare: Formula de actualizare pentru BGD: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$. Gradientul funcției de pierdere (MSE) în formă vectorială (conform cunoștințelor anterioare, bazate pe material):

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \frac{1}{n} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Pasul 1: Construirea matricelor $\mathbf{X}$ și $\mathbf{y}$ (n=3):

$$\mathbf{X} = [1 \quad 1\ 1 \quad 2\ 1 \quad 3], \quad \mathbf{y} = [1\ -1\ 2]$$

Pasul 2: Calculul predicțiilor inițiale $\hat{\mathbf{y}}^{(0)}$ și al erorii $\mathbf{e}^{(0)}$:

$$\hat{\mathbf{y}}^{(0)} = \mathbf{X}\mathbf{w}^{(0)} = [1 \quad 1\ 1 \quad 2\ 1 \quad 3]\,[0\ 0] = [0\ 0\ 0]$$

$$\mathbf{e}^{(0)} = \mathbf{X}\mathbf{w}^{(0)} - \mathbf{y} = [0\ 0\ 0] - [1\ -1\ 2] = [-1\ 1\ -2]$$

Pasul 3: Calculul gradientului $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}^{(0)})$:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}^{(0)}) = \frac{1}{3} \mathbf{X}^\top \mathbf{e}^{(0)} = \frac{1}{3} [1 \quad 1 \quad 1\ 1 \quad 2 \quad 3]\,[-1\ 1\ -2] = \frac{1}{3} [-1 + 1 - 2\ \ -1 + 2 - 6] = \frac{1}{3} [-2\ \ -$$

Pasul 4: Actualizarea ponderilor $\mathbf{w}^{(1)}$:

$$\mathbf{w}^{(1)} \leftarrow \mathbf{w}^{(0)} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}^{(0)})$$

$$\mathbf{w}^{(1)} \leftarrow [0\ 0] - 0.1\,[-0.667\ \ -1.667] = [0 + 0.0667\ 0 + 0.1667] \approx [0.067\ 0.167]$$

Exercițiul B.2: Varianța Algoritmilor de Coborâre a Gradientului

Comparați **Batch Gradient Descent (BGD)** și **Stochastic Gradient Descent (SGD)** din perspectiva utilizării datelor și a convergenței.

Rezolvare B.2:

1. **Utilizarea Datelor:**
   - **BGD** folosește **întregul set de antrenare** la fiecare pas de actualizare a gradientului, asigurând o direcție de coborâre precisă [precedent conversation].
   - **SGD** folosește doar **o singură instanță aleatorie** pentru a calcula gradientul, ceea ce îl face mult mai rapid per pas [precedent conversation].
2. **Convergența:**
   - **BGD** converge lin și precis la minimul (global) [precedent conversation].
   - **SGD** converge rapid spre minim, dar **progresează neregulat** (*bounces up and down*) și nu se așază niciodată exact la minim din cauza naturii aleatorii a estimării gradientului [precedent conversation].

C. Regresia Polinomială (Polynomial Regression)

Exercițiul C.1: Extinderea Caracteristicilor

Avem un set de date cu o singură caracteristică x. Dacă dorim să aplicăm Regresia Polinomială de **gradul** $M = 4$. Scrieți forma ecuației modelului $\hat{y}$ și vectorul de caracteristici transformat $\phi(x)$.

Rezolvare C.1:

1. **Ecuația Modelului:** Regresia Polinomială de grad 4 include puteri ale lui x până la 4 [precedent conversation].

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4$$

2. **Vectorul de Caracteristici Transformat $\phi(x)$:** Vectorul include termenul de bias (1) și puterile caracteristicii x până la gradul M [precedent conversation].

$$\phi(x) = [1 \ x \ x^2 \ x^3 \ x^4]$$

Exercițiul C.2: Identificarea Overfitting-ului

Explicați cum se manifestă **overfitting-ul** în contextul Regresiei Polinomiale, comparând eroarea de antrenare cu eroarea de testare.

Rezolvare C.2:

Overfitting-ul apare atunci când gradul polinomial M este prea mare (de exemplu, $M = 9$ pentru $n = 10$ puncte) [precedent conversation].

1. **Eroare de Antrenare:** Modelul de grad mare se potrivește perfect (sau aproape perfect) datelor de antrenare, rezultând o **eroare de antrenare foarte mică** [precedent conversation].
2. **Eroare de Testare:** Modelul **oscilează sălbatic** în afara datelor de antrenare, fiind excesiv de sensibil la zgomot. Ca rezultat, performanța pe datele noi (testare) este **mult mai slabă** (eroarea de testare este mare) [precedent conversation].

D. Modele Liniare Regularizate (Regularized Linear Models)

Exercițiul D.1: Diferența L1 vs L2 (Natura Ponderilor)

Explicați diferența fundamentală în ceea ce privește **efectul asupra ponderilor** între Regresia Ridge (ℓ_2) și Regresia Lasso (ℓ_1).

Rezolvare D.1:

1. **Ridge Regression (ℓ_2):** Aplică o penalizare pe norma ℓ_2 (suma pătratelor ponderilor). Aceasta forțează ponderile să fie **mici** (le micșorează spre zero), dar **nu le anulează** [precedent conversation].
2. **Lasso Regression (ℓ_1):** Aplică o penalizare pe norma ℓ_1 (suma valorilor absolute). Aceasta are proprietatea unică de a **seta la zero** ponderile caracteristicilor mai puțin importante, realizând astfel **selecția automată a caracteristicilor** (producând un model rar/sparse) [precedent conversation].

Exercițiul D.2: Controlul Regularizării

Care este rolul hiperparametrului λ în funcția de cost regularizată [precedent conversation]? Descrieți ce se întâmplă când λ tinde spre zero și când λ tinde spre infinit.

Rezolvare D.2:

Hiperparametrul λ (sau $r\lambda$ în Elastic Net [precedent conversation]) **controlează intensitatea regularizării** (sau compromisul dintre potrivirea datelor și menținerea ponderilor mici).

1. **Când $\lambda = 0$:** Termenul de regularizare dispare. Modelul revine la **Regresia Liniară simplă** (MLE) [precedent conversation].
2. **Când $\lambda \to \infty$:** Penalizarea domină. Modelul este forțat să aibă ponderi foarte mici (aproape zero), rezultând o linie plată care aproximează media datelor [precedent conversation].

E. Regresia Logistică (Logistic Regression)

Exercițiul E.1: Punctul de Decizie (Threshold)

Definiți funcția Sigmoid $\sigma(t)$ și explicați care este valoarea prag implicită a probabilității estimate $\hat{p}$ folosită pentru a clasifica o instanță în clasa pozitivă ("1") [precedent conversation].

Rezolvare E.1:

1. **Funcția Sigmoid (Logistică):** Este definită ca $\sigma(t) = \frac{1}{1+\exp(-t)}$, unde t este scorul liniar $\mathbf{w}^\top \mathbf{x}$ [precedent conversation]. Funcția mapează $t \in \mathbb{R}$ la $\hat{p} \in$.
2. **Pragul de Decizie:** Modelul clasifică o instanță ca aparținând clasei pozitive ("1") dacă probabilitatea estimată $\hat{p}$ este ≥ 0.5 (sau 50%) [precedent conversation].

Exercițiul E.2: Necesitatea Log Loss

De ce este necesară utilizarea funcției de pierdere **Log Loss (Binary Cross-Entropy)** pentru antrenarea Regresiei Logistice, și nu se poate folosi Eroarea Pătratică Medie (MSE), așa cum se face la Regresia Liniară?

Rezolvare E.2:

Log Loss este necesară deoarece:

1. Funcția Sigmoid introdusă în Regresia Logistică face ca suprafața de pierdere MSE să **nu fie convexă** [precedent conversation].
2. Dacă suprafața de pierdere nu este convexă, algoritmii de optimizare (precum Coborârea Gradientului) ar putea fi blocați în **minime locale**, neputând garanta găsirea soluției optime (minimul global) [precedent conversation].
3. Log Loss menține suprafața de pierdere **convexă**, garantând astfel că optimizarea atinge minimul global [precedent conversation].

F. Regresia Softmax (Softmax Regression)

Exercițiul F.1: Proprietățile Probabilităților Softmax

Presupunem că un model Softmax estimează probabilitățile $\hat{p}_k$ pentru 4 clase.

1. Care este valoarea totală la care trebuie să însumeze probabilitățile estimate de Softmax (adică $\sum_{k=1}^{4} \hat{p}_k$)?
2. Dacă scorurile calculate sunt $o_1 = 5, o_2 = 5, o_3 = 0, o_4 = 0$, ce probabilitate estimată $\hat{p}_1$ ați aștepta să obțineți?

Rezolvare F.1:

1. Softmax transformă scorurile în probabilități **normalizate**; prin urmare, suma probabilităților estimate pentru toate clasele trebuie să fie întotdeauna **1** [precedent conversation].
2. Dacă scorurile o_1 și o_2 sunt egale și celelalte scoruri sunt mai mici, probabilitățile $\hat{p}_1$ și $\hat{p}_2$ vor fi egale și mai mari decât $\hat{p}_3$ și $\hat{p}_4$. Deoarece $\sum_{k=1}^{4} \hat{p}_k = 1$, probabilitățile se vor distribui conform funcției exponențiale.

Exercițiul F.2: Decizia de Clasificare Softmax

Un clasificator Softmax a calculat scorurile (logits) pentru trei clase: $o_1 = -1.5$, $o_2 = 3.2$, $o_3 = 1.0$.

1. Care este clasa prezisă de model?
2. Explicați de ce nu este necesar să calculăm efectiv probabilitățile $\hat{p}_k$ (folosind funcția Softmax) pentru a lua decizia de clasificare finală.

Rezolvare F.2:

1. **Clasa Prezisa:** Clasificatorul Softmax prezice clasa care are **scorul cel mai mare** [precedent conversation].

$$\hat{y} = \arg\max_k o_k(\mathbf{x})$$

Dintre scorurile $-1.5, 3.2, 1.0$, scorul maxim este **3.2** (corespunzător clasei 2). Prin urmare, modelul prezice **Clasa 2**.

2. **Necesitatea Calculului:** Nu este necesar să calculăm efectiv probabilitățile Softmax, deoarece funcția Softmax este **monotonă** (adică păstrează ordinea scorurilor). Clasa cu cel mai mare scor o_k va avea întotdeauna și cea mai mare probabilitate estimată $\hat{p}_k$ [precedent conversation].